

EXPERIMENT 1

Introduction to Maven and Gradle: Overview of Build Automation Tools, Key Differences Between Maven and Gradle, Installation and Setup.

1. Overview of Build Automation Tools:

Build automation tools simplify the process of **compiling, testing, packaging, and deploying** software projects. They manage dependencies, execute tasks, and integrate seamlessly with CI/CD pipelines.

Build automation tools manage the entire build lifecycle:

- **Dependency Management:** Automatically identify, download, and manage external libraries and their transitive dependencies required by the project.
- **Compilation:** Transform human-readable source code (e.g., Java, C++) into machine code or bytecode.
- **Testing:** Automatically run unit and integration tests to ensure code quality and detect regressions early.
- **Packaging:** Bundle the compiled code and resources into a distributable format (e.g., JAR, WAR files, Docker images).
- **Publishing:** Push the final artifact to a central repository for deployment to various environments (development, staging, production).

Key Build Automation Tools

Tools can be broadly classified into build automation utilities (which perform the actual build tasks) and build automation servers (which orchestrate the CI/CD pipeline).

Build Utilities

These tools are some commonly used tools which are language-specific or multi-language and focus on the mechanics of building the software:

Tool	Common Use Case
Apache Maven	Java enterprise projects, standardized builds
Gradle	Android development, modern JVM and multi-language projects
Bazel	Large distributed systems, multi-language repositories
Make / CMake	C/C++ projects, cross-platform builds
Apache Ant	Legacy Java projects, custom build processes
MSBuild	.NET and Windows development

Why Use Build Automation?

1. Ensures **consistency** across builds.
2. Handles **dependency management** automatically.
3. Reduces **manual errors** and increases efficiency.
4. Integrates with tools like **Jenkins & Azure DevOps**.

2. Key Differences Between Maven and Gradle:

Maven is great for structured, **enterprise-level** projects.

Gradle is ideal for **scalable and performance-driven** applications.

Feature	Maven (XML)	Gradle (Groovy/Kotlin)
Build Script	pom.xml	build.gradle / build.gradle.kts
Performance	Sequential, slower	Parallel execution, faster
Flexibility	Convention-based	Highly customizable
Dependency Management	Uses Maven Repository	Supports multiple repositories
Ease of Use	Simple XML structure	Slightly complex but powerful
Caching Support	No build caching	Supports incremental builds
Best For	Standard Java projects	Complex, high-performance builds

3. Installation and Setup

3.1 Installing Maven for windows

Step 1: Install Java (JDK 17 Recommended)

Check if Java is installed:

```
1 java -version
2 javac -version
3
```

Step 2: Download and Install Maven

Download from: [Download Apache Maven – Maven](#)

Extract it to a folder (e.g., C:\Maven).

Step 3: Configure Environment Variables

Add MAVEN_HOME → C:\Maven\apache-maven-<version>
Update Path → %MAVEN_HOME%\bin

Step 4: Verify Installation

Run:

```
1 mvn -version
2
```

Fig 1.2: Maven Verify Installation

Expected Output:

```
1 Apache Maven 3.x.x
2 Maven home: C:\Maven\apache-maven-<version>
3 Java version: 17.0.4
4
```

3.2 Installing Gradle for windows**Step 1: Install Java (JDK 17 Recommended)**

Same steps as Maven.

Download from: <https://gradle.org/releases/>

Extract it to a folder (e.g., C:\Gradle).

Step 2: Configure Environment Variables

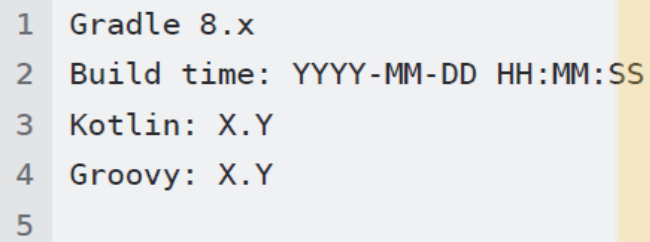
Add GRADLE_HOME → C:\Gradle\gradle-<version>
Update Path → %GRADLE_HOME%\bin

Step 3: Verify Installation

Run:

```
1 gradle -v
2
```

Expected Output:



```
1 Gradle 8.x
2 Build time: YYYY-MM-DD HH:MM:SS
3 Kotlin: X.Y
4 Groovy: X.Y
5
```

Fig 1.5: Gradle Installation Output

EXPERIMENT 2

Working with Maven: Creating a Maven Project, Understanding the POM File, Dependency Management and Plugins.

Introduction to Maven

Maven is a build automation tool primarily used for Java projects. It simplifies the build process, manages project dependencies, and supports plugins for different tasks. It uses XML files (called pom.xml) for project configuration and dependency management.

Key Benefits of Maven:

1. Dependency management (automates downloading and including libraries).
2. Build automation (compiles code, runs tests, creates artifacts).
3. Consistent project structure (standardizes how Java projects are set up).
4. Integration with CI tools (like Jenkins).

Key Features of Maven:

- **Project Management:** Handles project dependencies, configurations, and builds.
- **Standard Directory Structure:** Encourages a consistent project layout across Java projects.
- **Build Automation:** Automates tasks such as compilation, testing, packaging, and deployment.
- **Dependency Management:** Downloads and manages libraries and dependencies from repositories (e.g.,Maven Central).
- **Plugins:** Supports many plugins for various tasks like code analysis, packaging, and deploying.

1. Creating a Maven Project

Steps to Create a Maven Project in IntelliJ IDEA

1.1 Install Maven (if not already installed):

In command prompt check mvn -version to check if Maven installed or not.

1.2. Create a New Maven Project:

- Open **IntelliJ IDEA**.
- Go to **File > New > Project** .
- Set the **project name** and **location** . (make sure to choose location other than C drive)
- Select Maven from the **project Build system**.
- You can specify the **GroupId** as for example kvgce_Aiml or any other organization name.(optional)
- click **Create** .

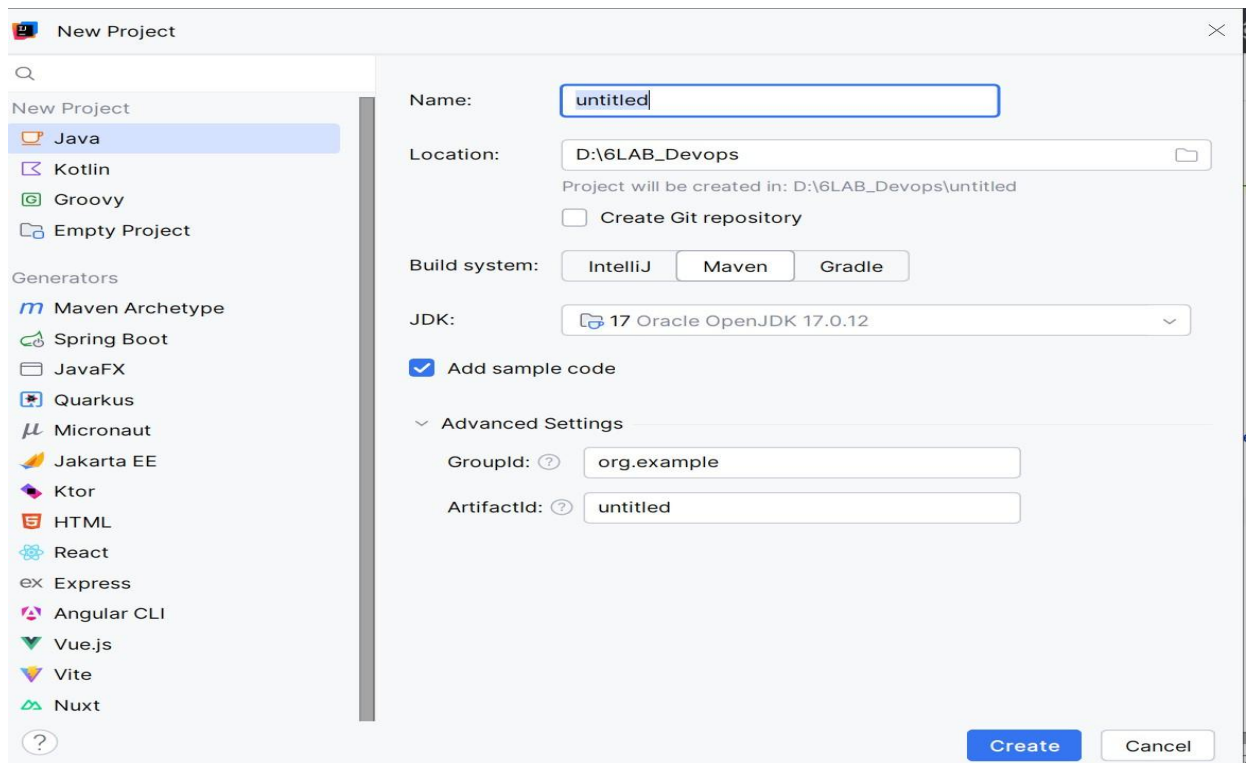


Fig 2.1: Creation of new Maven project on IntelliJ IDEA

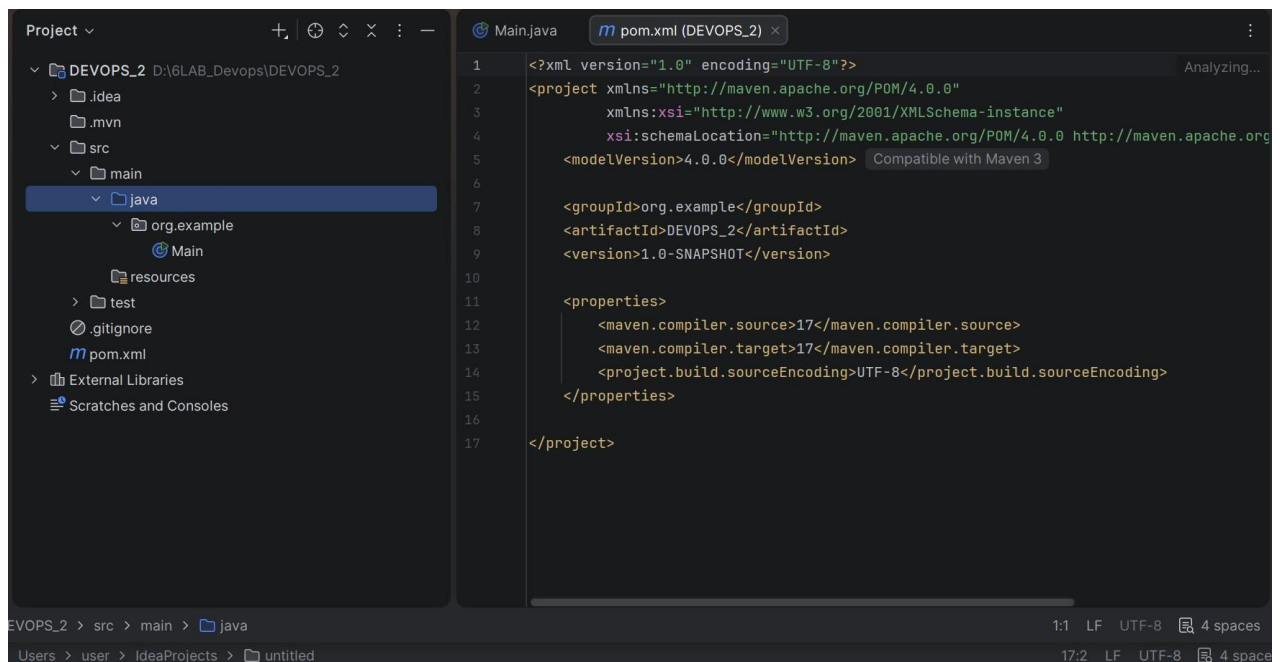


Fig 2.2: After Creating a Maven project

2. Understanding the POM File

In Apache Maven, the **POM** stands for **Project Object Model**. It is an **XML file (pom.xml)** that contains information about the project and configuration details used by Maven to build the project.

The pom.xml file:

- Defines project details (name, version, packaging type).
- Manages dependencies (external libraries).
- Configures plugins.
- Controls build lifecycle.
- Handles project structure.

The newly created project will have a pom.xml file in its root directory. This file is crucial for Maven configuration. This XML file includes project configuration details, dependencies, and plugins.

Example of a basic pom.xml file:

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
3         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
5     <modelVersion>4.0.0</modelVersion> Compatible with Maven 3
6
7     <groupId>org.example</groupId>
8     <artifactId>DEVOPS_2</artifactId>
9     <version>1.0-SNAPSHOT</version>
10
11     <properties>
12         <maven.compiler.source>17</maven.compiler.source>
13         <maven.compiler.target>17</maven.compiler.target>
14         <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
15     </properties>
16
17 </project>
```

Fig 2.3: Example of a basic pom.xml file

Important Elements in pom.xml

<modelVersion>

- Defines POM model version.
- Usually 4.0.0.

<groupId>

- Organization or company name.
- Example: com.companyname

Similar to Java package naming convention.

<artifactId>

- Project name.
- Example: student-management-system

<version>

- Project version.
- Example: 1.0.0, 1.0-SNAPSHOT

SNAPSHOT → Development version.

<packaging>

Specifies output type:

Packaging Type	Output
jar	Java Archive
war	Web Application Archive
ear	Enterprise Archive

3. Dependency Management and Plugins.

3.1 Dependency Management

Dependency management is the process by which **Maven automatically downloads, updates, and manages external libraries** required by a project.

Maven automatically downloads required libraries from repositories.

XML

```
<dependencies>
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-core</artifactId>
    <version>5.3.9</version>
  </dependency>
</dependencies>
```

Fig 2.4: Adding Dependencies from Maven Repository

Maven downloads from **Maven Central Repository**

3.2 Add Dependencies for TestNG:

Open **pom.xml**.

- To add a dependency, find the dependency information on Maven Repository (e.g., <https://mvnrepository.com/>). In search tab type TestNG. Click on the first result.

TestNG

TestNG is a testing framework inspired from JUnit and NUnit but introducing some new functionalities that make it more powerful and easier to use. It supports test configured by annotations, data-driven testing, parametric tests, etc.

Overview Versions (123) Used By (12.9k) BOMs (309) Badges Books (15)

LICENSE Apache 2.0

CATEGORIES Testing

TAGS testing testng quality

RANKING #44 in MvnRepository #8 in Testing

HOMEPAGE <https://testng.org> [Inspect URL](#)

LINKS [GitHub](#)

SPONSORED

Secure JS file uploader. Easy setup | User Friendly | Reliable. [Start Free](#).

Ads by EthicalAds

Latest Versions 123 versions →

VERSION	VULNERABILITIES	USAGES	DATE
7.12.x	7.12.0	82	Jan 22, 2026
7.11.x	7.11.0	451	Feb 13, 2025
	7.10.2	632	Apr 28, 2024

Fig 2.5: Searching TestNG on Maven Repository

In latest version grid choose the most recent version.

Home » org.testng » testng » 7.12.0

TestNG » 7.12.0

TestNG is a testing framework inspired from JUnit and NUnit but introducing some new functionalities that make it more powerful and easier to use. It supports test configured by annotations, data-driven testing, parametric tests, etc.

Overview Dependencies (5) Changes (0) Books (15)

LICENSE Apache 2.0

CATEGORIES Testing

TAGS testing testng quality

HOMEPAGE <https://testng.org> [Inspect URL](#)

LINKS [🔗](#)

DATE Jan 22, 2026

FILES pom (2 KB) jar (969 KB) [View All](#)

REPOSITORIES Central

RANKING #44 in MvnRepository #8 in Testing

SPONSORED

One API to manage all your file uploads. Effortless integration: Reliable performance [Start Free](#)

Ads by EthicalAds

Maven Gradle SBT Mill Ivy Grape Leiningen Buildr

Scope: Test [Include backlinks](#)

```
<!-- Source: https://mvnrepository.com/artifact/org.testng/testng -->
<dependencies>
```

Fig 2.6: Selecting the most recent version of TestNG

Under the second tab by default Maven code is present. If you click inside the code box, the code will be automatically copied to clipboard. Now comeback to POM.xml in IntelliJ IDEA and copy it under **dependencies** tag.

```
6      <groupId>com.example</groupId>
7      <artifactId>simple-project</artifactId>
8      <version>1.0-SNAPSHOT</version>
9
10     <dependencies>
11         <!-- Add your dependencies here -->
12     </dependencies>
13
14 </project>
```

Fig 2.7: Copying dependency code to POM.xml in IntelliJ IDEA

```

16
17     <dependencies>
18         <!-- Add your dependencies here -->
19         <!-- Source: https://mvnrepository.com/artifact/org.testng/testng -->
20         <dependency>
21             <groupId>org.testng</groupId>
22             <artifactId>testng</artifactId>
23             <version>7.12.0</version>
24             <scope>test</scope>
25         </dependency>
26     </dependencies>

```

Fig 2.8: TestNG Dependency added in pom.xml

Note: In a Maven pom.xml file, the <dependencies> tag should typically be placed **after** the <properties> tag.

3.3 Plugins

Plugins are tools that perform specific tasks during the build life

cycle. It performs the following activities

- compile code
- Run tests
- Package applications
- Deploy artifacts
- Generate reports

Rank	Plugin Name	Purpose
1	maven-compiler-plugin	Compiles Java source code
2	maven-surefire-plugin	Runs unit tests
3	maven-clean-plugin	Cleans old build files
4	maven-jar-plugin	Packages project into JAR
5	maven-install-plugin	Installs artifact into local repo

6	maven-deploy-plugin	Deploys artifact to remote repo
7	maven-war-plugin	Builds WAR files for web apps
8	maven-dependency-plugin	Manages dependencies
9	maven-failsafe-plugin	Runs integration tests
10	maven-site-plugin	Generates project documentation

Build plugins are executed during the project's build lifecycle (e.g., compilation, testing, packaging). The **<plugin>** tag should be placed inside the **<plugins>** sub-element within the **<build>** section.

```
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.11.0</version>
      <configuration>
        <source>1.8</source>
        <target>1.8</target>
      </configuration>
    </plugin>
  </plugins>
</build>
```

Fig 2.9: Adding Plugins inside the build section

You can read more about this on : <https://maven.apache.org/plugins/index.html>

Key Difference between Dependency Management and Plugins

Aspect	Dependency Management	Plugins
Purpose	Manages libraries	Performs build tasks
Function	Adds external code	Executes operations
Example	JUnit, Spring	Compiler, Surefire
Needed for	Code functionality	Build automation

EXPERIMENT 3

Working with Gradle: Setting Up a Gradle Project, Understanding Build Scripts (Groovy and Kotlin DSL), Dependency Management and Task Automation.

1. Introduction to Gradle

Gradle is a powerful **build automation tool** used mainly for Java projects. It helps automate tasks such as compiling code, running tests, packaging applications, and managing dependencies.

Gradle uses **build scripts** to define how a project should be built. These scripts are written using a **Domain Specific Language (DSL)**.

Gradle supports **two scripting languages** for writing build scripts:

- **Groovy DSL** – Uses the .gradlefile extension and is the traditional, dynamically typed scripting language used in Gradle.
- **Kotlin DSL** – Uses the .gradle.kts file extension and is a modern, statically typed alternative that provides **better IDE support, improved code completion, and compile-time error checking**.

Gradle build scripts are organized using **script blocks such as plugins {}, repositories {}, and dependencies {}**, which help configure plugins, define dependencies, and manage the build process efficiently.

DSL Type	File Name	Language Used
Groovy DSL	build.gradle	Groovy
Kotlin DSL	build.gradle.kts	Kotlin

2. Dependency Management

Dependency management allows Gradle to automatically download external libraries required for the project.

Configuration	Purpose
Implementation	Required for compilation and runtime
Api	Exposed to dependent projects
testImplementation	Used for testing
runtimeOnly	Needed only during runtime

Gradle automatically downloads dependencies from **Maven Central Repository**.

3. Setting Up a Gradle Project

3.1 Verify Gradle Installation

Open the terminal or command prompt and type: `gradle -v`

If Gradle is installed successfully, the version details will be displayed.

3.2 Create a new Gradle project in IntelliJ IDEA

1. Open **IntelliJ IDEA**
2. Click **File** → **New** → **Project**
3. Select **Java Project**
4. Enter the **Project Name**
5. Choose the **Project Location**
6. Select **Build System: Gradle**
7. Select **Gradle DSL: Groovy**
8. Click **Create**

IntelliJ IDEA automatically creates the project structure and the **build.gradle** file.

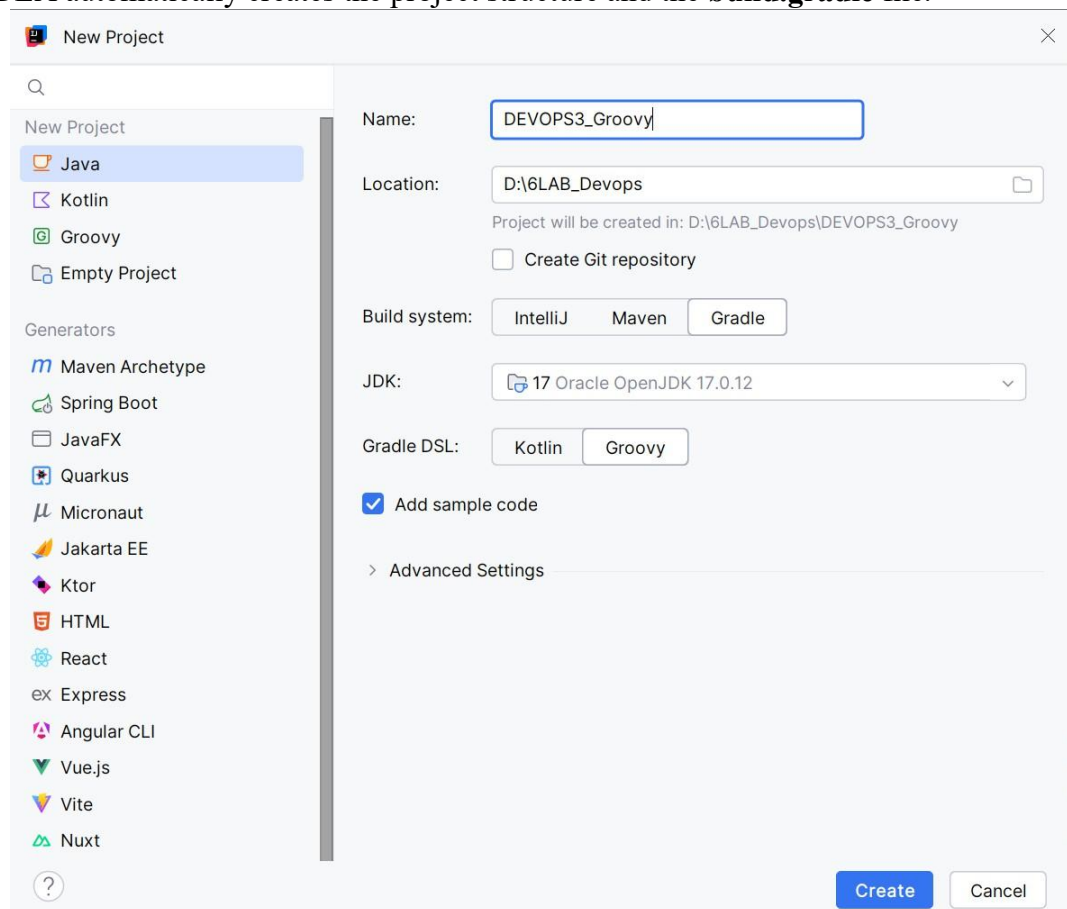


Fig 3.1: Setting up Gradle build file

3.3 Project Folder Structure

The typical Gradle project structure looks like this:

project-name

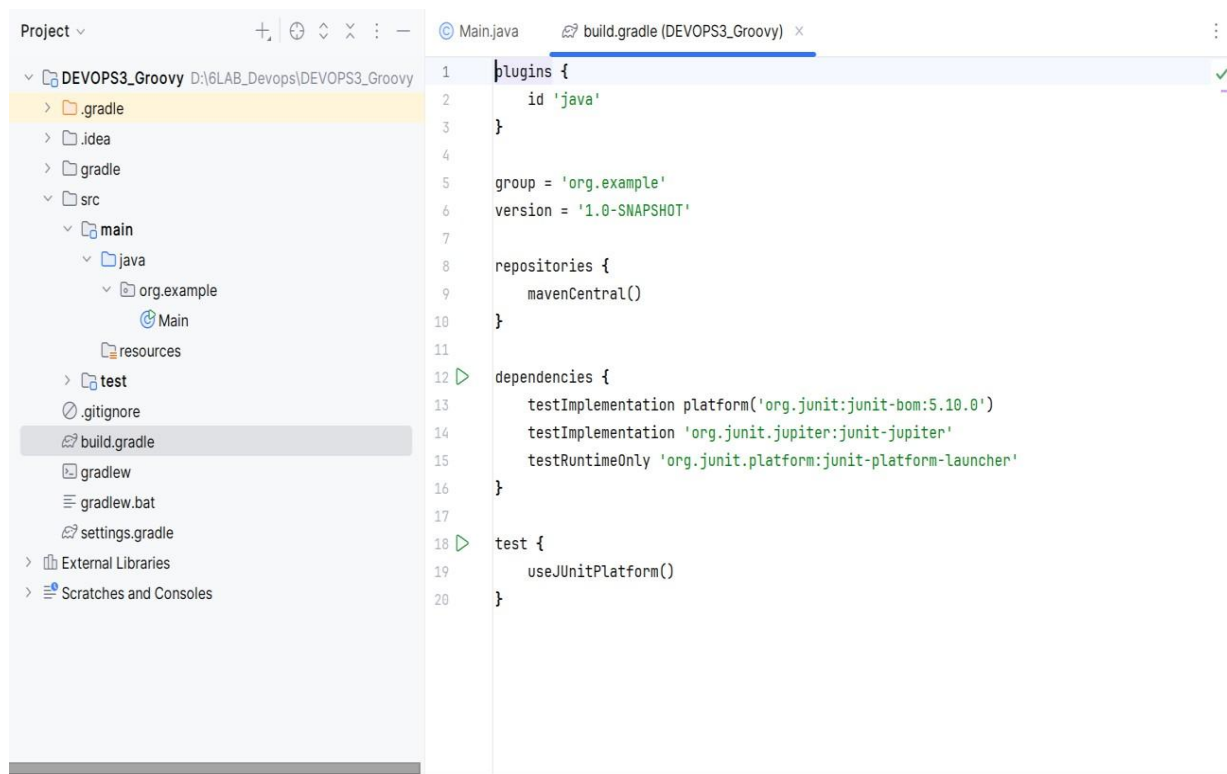
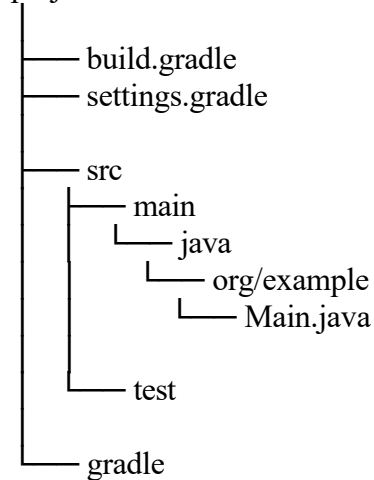


Fig 3.2: Gradle project directory structure

3.4 Modify the build.gradle File



Fig 3.3: build.gradle File configuration

3.5 Create Java Program

Create **Main.java** inside: src/main/java/org.example

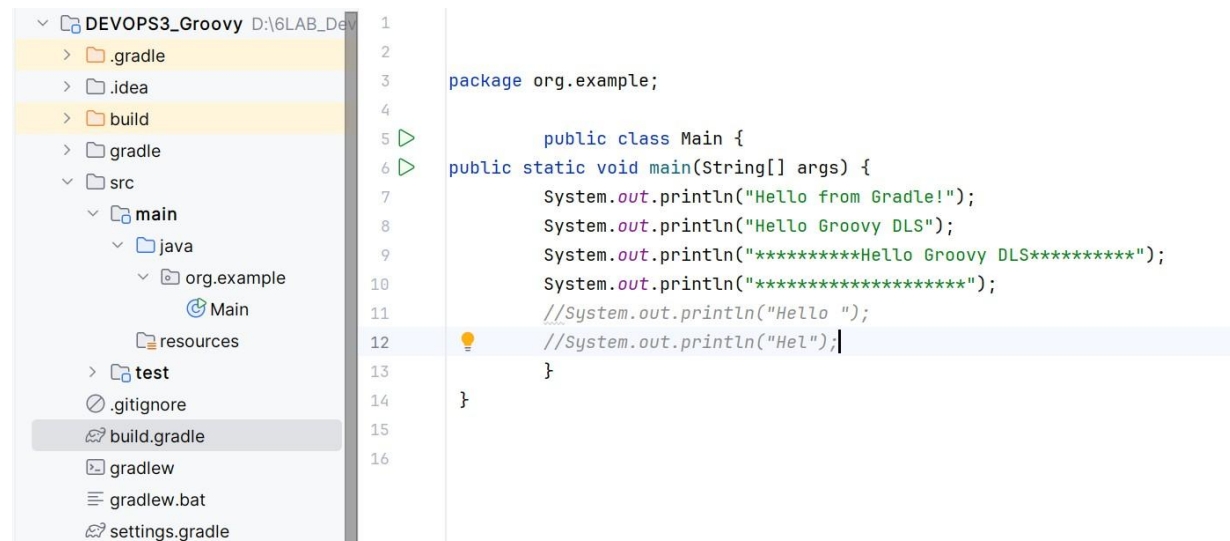


Fig 3.4: Main.java inside src/main/java/org.example

3.6 Build and Run the Project

Using IntelliJ IDEA

1. Open **Gradle Tool Window**
2. Navigate to

Tasks → application → run

3. Double click **run**

Using Terminal

```
gradle run
```

4. Adding Dependencies

1. Visit the website <https://mvnrepository.com>
2. Search for the required library.
3. Select the latest version.
4. Copy the **Gradle (Groovy Short.)** dependency code.



Fig 3.5: Adding dependency code in Gradle

Add it inside the **dependencies block** in the **build.gradle** file.

After adding the dependency:

1. IntelliJ automatically imports it
OR
2. Click **Load Gradle Changes**

```
dependencies {  
    testImplementation 'org.junit.jupiter:junit-jupiter:5.8.1'  
  
    // Source: https://mvnrepository.com/artifact/com.google.guava/guava  
    implementation 'com.google.guava:guava:33.5.0-jre'  
}  
  
application {  
    //mainClass = 'com.example.Main'  
    mainClass = 'org.example.Main'  
}
```

Fig 3.6: Sync Gradle Changes

5. Task Automation in Gradle

A **task** represents an independent unit of work performed during the build process.

Examples include:

- Compiling source code
- Running tests
- Creating JAR files
- Packaging applications
- Managing dependencies

Task automation helps automate repetitive development activities using Gradle's **Groovy-based DSL**.

5.1 Example: Creating a Custom Task

Add the following code at the bottom of **build.gradle**:

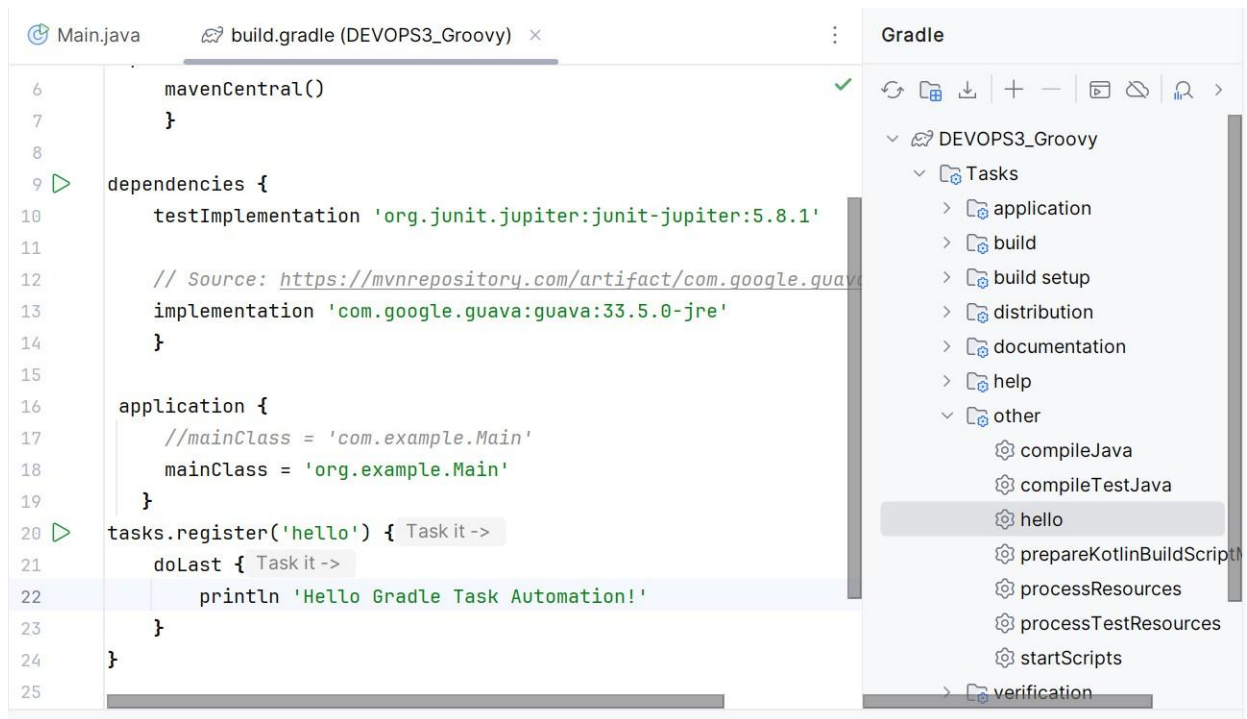


Fig 3.7: Running Gradle tasks

5.2 Running the Task

Using IntelliJ IDEA

1. Open **Gradle Tool Window**
2. Expand Tasks ☐ other.
3. Double click **hello**

OR Using terminal run command `./gradlew hello`

5.3 Automating Tasks:

Automating tasks means configuring tasks so that one task automatically triggers another.

Gradle automates common development tasks such as compilation, testing, packaging, and deployment.

Task	Description
build	Compiles and tests the project
clean	Removes build files
test	Runs test cases
jar	Creates a JAR file

```
tasks.register('task1') { Task it ->
    doLast { Task it ->
        println "Task 1 executed"
    }
}

tasks.register('task2') { Task it ->
    doLast { Task it ->
        println "Task 2 executed"
    }
}

tasks.register('task3') { Task it ->
    doLast { Task it ->
        println "Task 3 executed"
    }
}

task3.dependsOn task2
task2.dependsOn task1
|
```

Fig 3.8: Gradle build output

Running **task3** will execute:

task1 → task2 → task3

EXPERIMENT 4

Practical Exercise: Build and Run a Java Application with Maven, Migrate the same Application to Gradle.

Part 1: Create and Build a Java Application with Maven

Step 1: Create Maven Project in IntelliJ IDE

1. Open **IntelliJ IDEA**
2. Click **New Project**
3. Select **Maven**
4. Choose **Create from Archetype**
5. Select **maven-archetype-quickstart**
6. Click **Next**
7. Click **Finish**

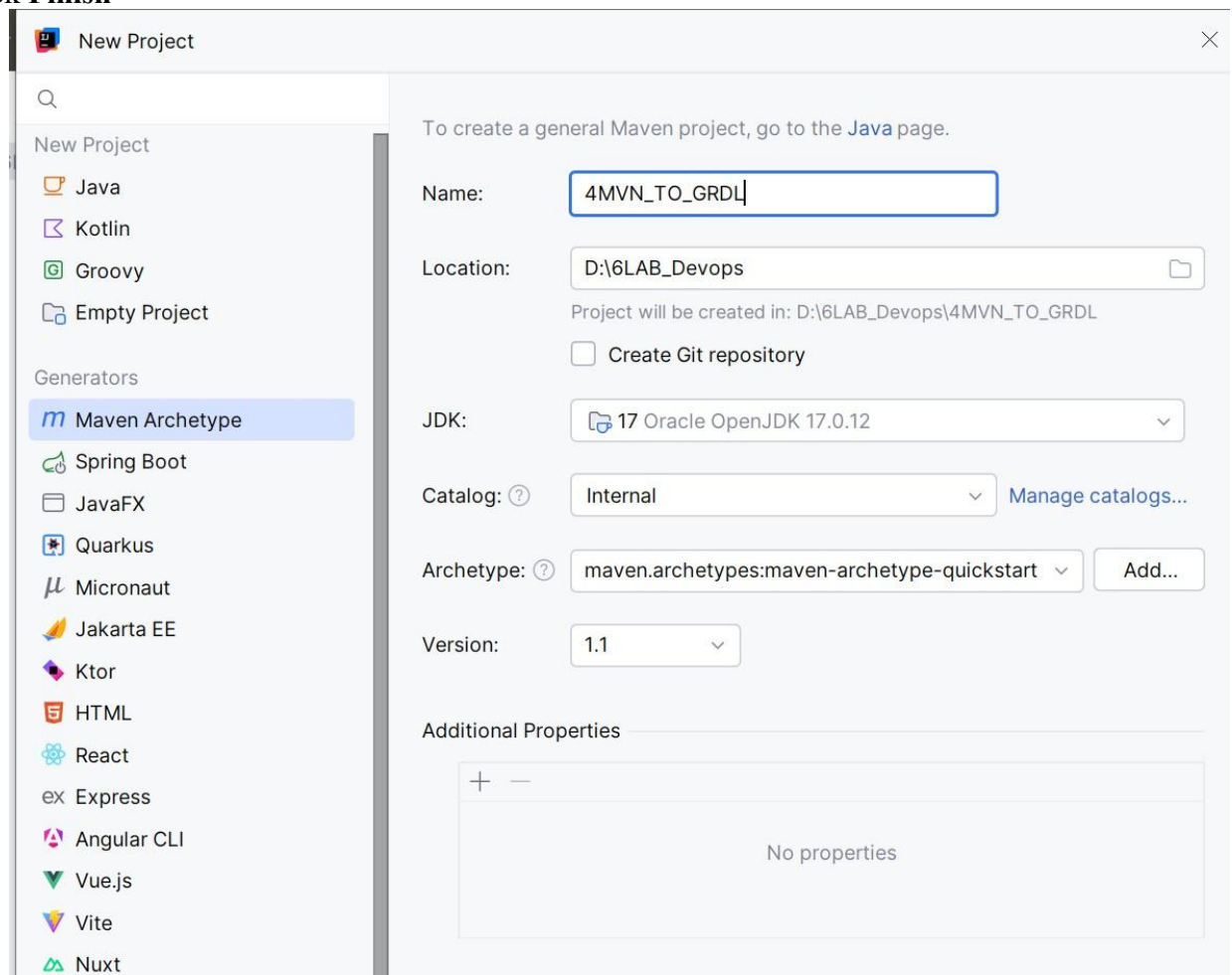


Fig 4.1: Creating new project - Click Finish

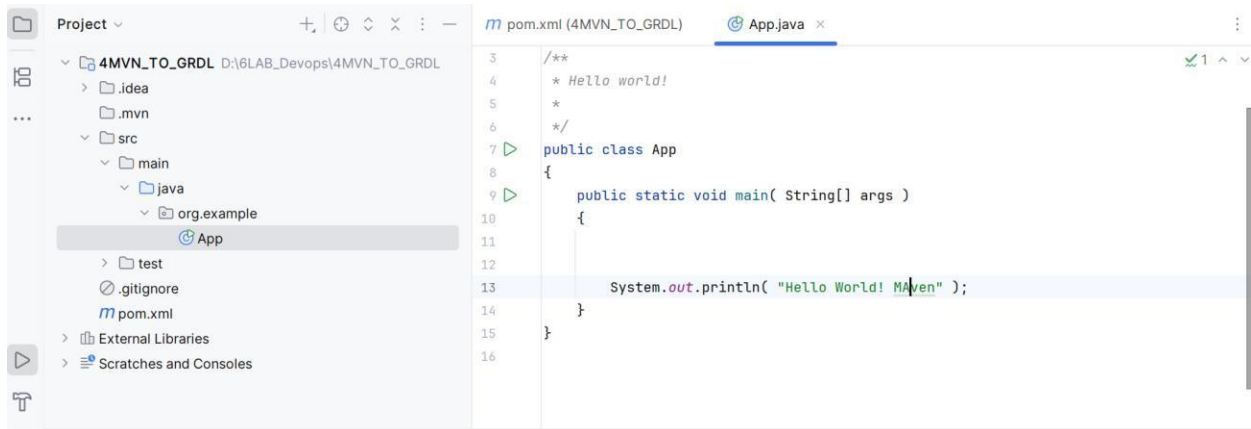
Step 2: add the Java code in src/main/java/org/example/App.java

Fig 4.2: Adding Java code in src/main/java/org/example/App.java

Step 3: Build and Run the Maven Project**a. Open IntelliJ IDEA Terminal**

Press **Alt + F12** to open the terminal or double click on terminal symbol.

b. Compile and Package the Project

Run the following commands to build the project one

after another. mvn compile

mvn package

you should get build success message.

c. Locate the JAR File

After the compile and package project, a .jar file will be located at: target/ProjectName-1.0-SNAPSHOT.jar

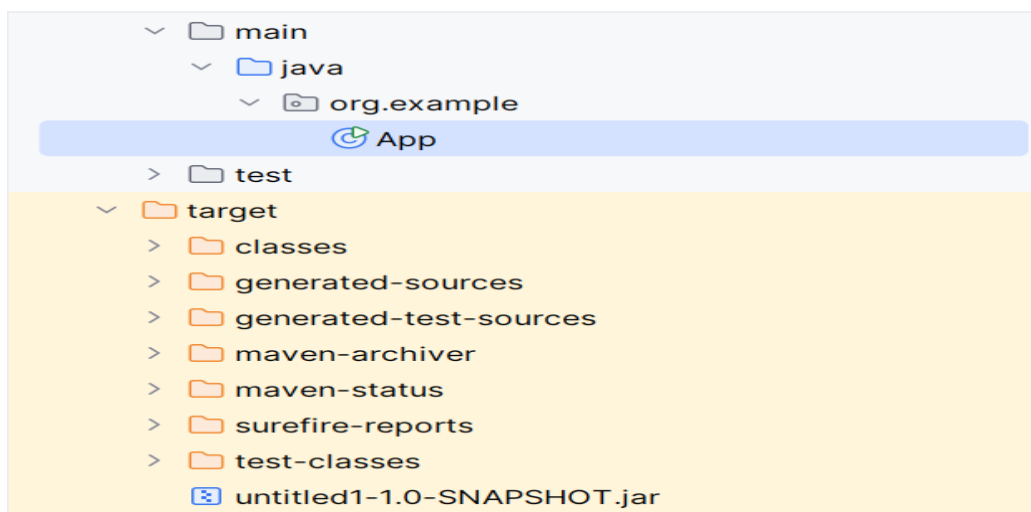


Fig 4.3: Maven build output at target directory

d. Run the Application

Run using IntelliJ:

Right Click → **App.java** → **Run**

Output: Hello World maven

Part 2: Migrate Maven Project to Gradle Step 1: Initialize Gradle in Your Project**a. Open Terminal in IntelliJ IDEA**

Make sure you're in the project directory:

```
cd
D:\6LAB_Devops\project_name>
```

b. Run Gradle Init Command

Execute the following command to migrate your Maven

project to Gradle: `gradle init --type pom`

This command will convert your Maven pom.xml into a Gradle build.gradle file.

```
PS D:\6LAB_Devops\untitled1> gradle init --type pom
Calculating task graph as no cached configuration is available for tasks: init --type pom

Select build script DSL:
  1: Kotlin
  2: Groovy
Enter selection (default: Kotlin) [1..2] 2

Generate build using new APIs and behavior (some features may change in the next minor release)? (default: no) [yes, no] no

> Task :init
Maven to Gradle conversion is an incubating feature.
For more information, please refer to https://docs.gradle.org/9.3.1/userguide/migrating\_from\_maven.html in the Gradle documentation.

BUILD SUCCESSFUL in 1m 12s
1 actionable task: 1 executed
Configuration cache entry stored.

PS D:\6LAB_Devops\untitled1>
```

Fig 4.4: Editing build.gradle file

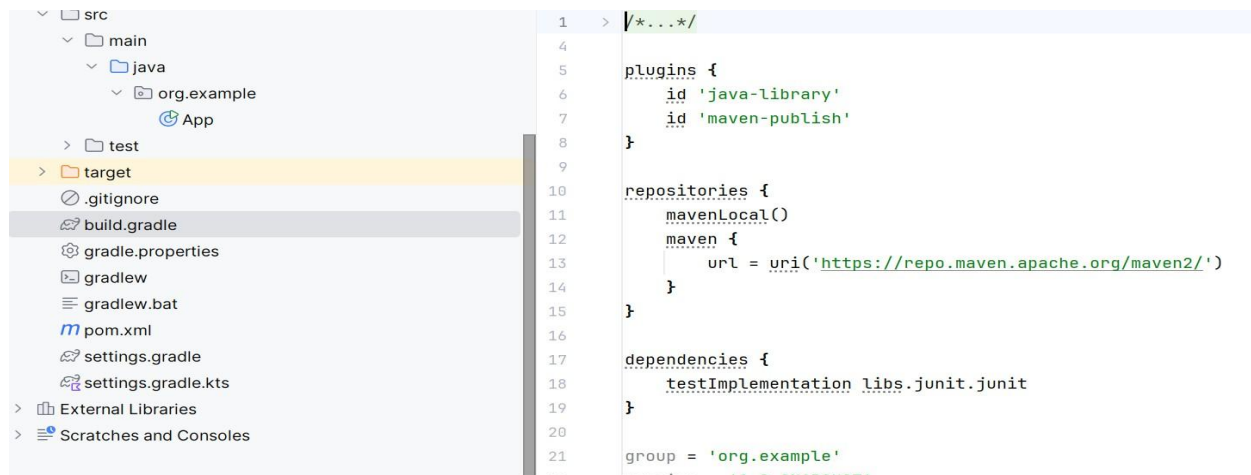


Fig 4.5: Gradle build.gradle configuration

Step 2: Review and Update build.gradle

1. Open build.gradle in IntelliJ IDEA.
2. Ensure the following configurations are correct:

changes required in

```

gradle.build file plugins {
    id 'java' //add this line to the existing line
}
  
```

```

repositories {
    mavenCentral() // fully change the repository
}
  
```

```

dependencies {
    testImplementation 'junit:junit:4.13.2' // change the syntax
}
  
```

//add these code into existing build.gradle

```

jar {
    manifest {
        attributes(
            'Main-Class': 'org.example.App'
        )
    }
}
  
```




```
1 > /*...*/
6
7 plugins {
8     id 'java-library'
9     id 'maven-publish'
10    id 'java'
11 }
12
13 repositories {
14     mavenCentral()
15 }
16
17
18 dependencies {
19     testImplementation 'junit:junit:4.13.2'
20 }
21
22 group = 'org.example'
23 version = '1.0-SNAPSHOT'
24 description = '4MVN_TO_GRDL'
```

Fig 4.6: Gradle build script configuration



```
38
39 tasks.withType(Javadoc) {
40     options.encoding = 'UTF-8'
41 }
42 jar {
43     manifest {
44         attributes(
45             'Main-Class': 'org.example.App'
46         )
47     }
48 }
49
```

Fig 4.7: Gradle dependencies configuration

Step 3: Build and Run the Gradle Project

a. At terminal run:

gradle build Successful

run generates a folder:

build/

Under that a Jar file: build/libs/GradleDemo-1.0-SNAPSHOT.jar must be present

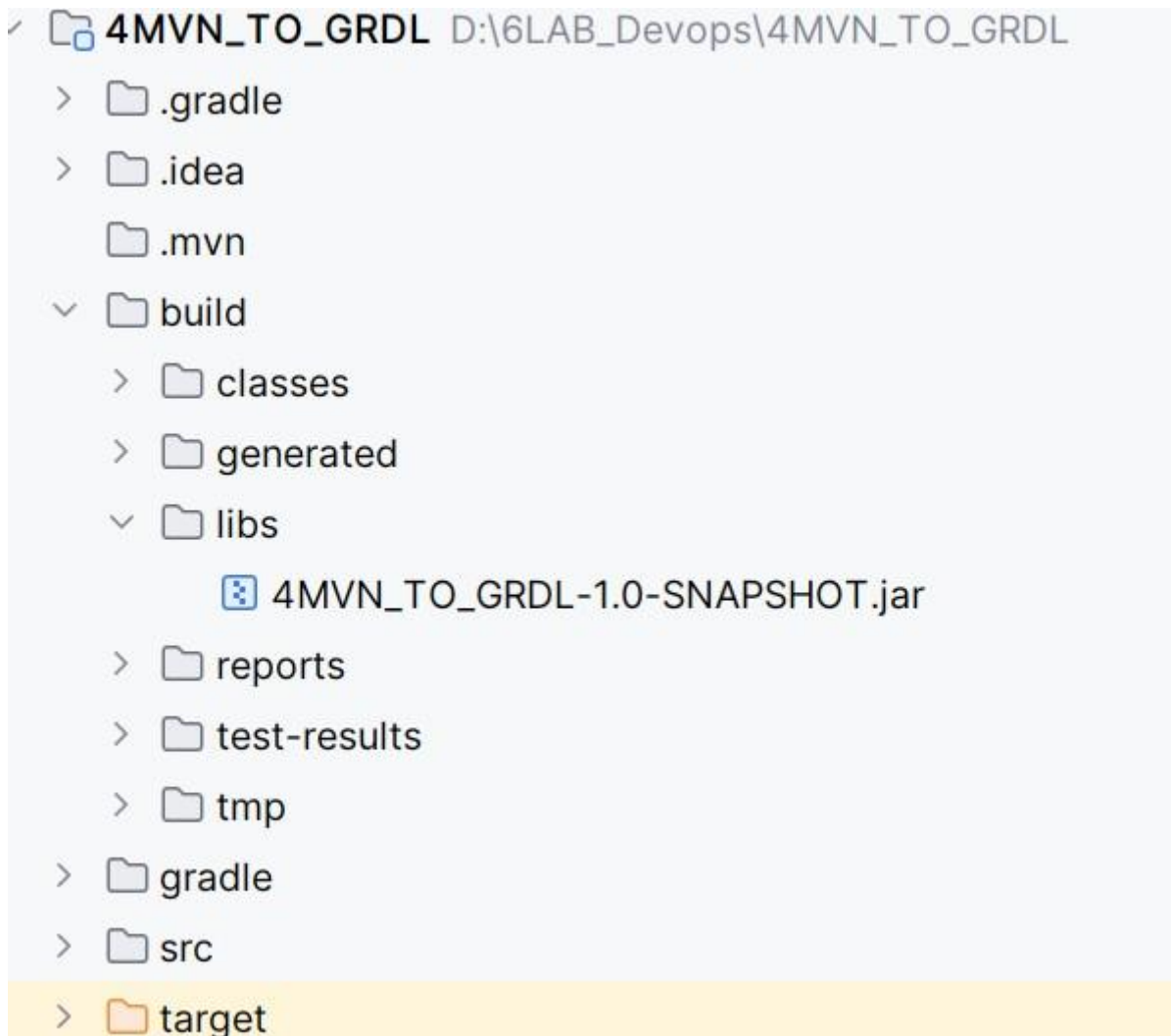


Fig 4.8: Verifying JAR must be present

b. run the gradle project:

java -jar build/libs/YourPROJECT Name-1.0-SNAPSHOT.jar

```
PS D:\6LAB_Devops\4MVN_TO_GRDL> gradle clean build
Calculating task graph as configuration cache cannot be reused because file 'build.gradle' has changed.

BUILD SUCCESSFUL in 2s
5 actionable tasks: 2 executed, 3 from cache
Configuration cache entry stored.

PS D:\6LAB_Devops\4MVN_TO_GRDL> java -jar build/libs/4MVN_TO_GRDL-1.0-SNAPSHOT.jar
Hello World! MAVen

PS D:\6LAB_Devops\4MVN_TO_GRDL> 
```

Fig 4.9: Gradle build output at build/libs directory

EXPERIMENT 5

Introduction to Jenkins: What is Jenkins?, Installing Jenkins on Local or Cloud Environment, Configuring Jenkins for First Use.

1.What Is Jenkins?

Jenkins is an open-source automation server written in Java. It is widely used to facilitate Continuous Integration (CI) and Continuous Deployment/Delivery (CD) pipelines in software development. Jenkins automates repetitive tasks related to building, testing, and deploying software, helping teams to integrate changes continuously and deliver high-quality applications faster.

CI (Continuous Integration): The process where developers frequently integrate their code changes into a shared repository, triggering automated builds and tests to ensure code stability.

CD (Continuous Delivery and Continuous Deployment): The phases following CI, where the tested code is prepared for release (delivery) and automatically deployed to production environments.

CI/CD Pipeline Workflow

1. Plan: Initial idea and planning of the software.
2. Code: Developers write and commit code to a source control repository like GitHub using tools like Git.
3. Build: Automated build tools (e.g., Maven, Gradle) compile the code.
4. Test: Automated testing frameworks (e.g., JUnit, TestNG) validate the build.
5. Release & Deploy: The tested and stable code is released and deployed to production.
6. Monitor & Feedback: Continuous monitoring ensures quality and improvement through feedback.

Detailed CI Process Breakdown

Step	Description	Tools/Frameworks
Code Commit	Developers push code changes to a central repository to trigger the pipeline.	Git, GitHub, GitLab, Bitbucket
Build	Source code is compiled and converted into executable artifacts.	Maven, Gradle, npm, Webpack
Test	Automated unit and integration tests run to ensure code functionality.	JUnit, TestNG, PyTest, Mocha
Code Quality Scans	Static analysis to find bugs, technical debt, and security vulnerabilities.	SonarQube, BlackDuck, Snyk
Reporting & Upload	Test results are logged, and successful builds are stored in an artifact repository.	JFrog Artifactory, Nexus, Allure

Step 1.Installing Jenkins

1.Prerequisites

- Ensure the java(JDK) installed on your system

2.Install Jenkins on window system

- Official Jenkins website
- Download Jenkins.war file .
- Goto cmd prompt and run command

java -jar jenkins.war --httpPort=8080

- After successfully installed, Jenkins will be running on port

http://localhost:8080 or 8088

3. Next Step (First-Time Setup)

Open the browser and go to the URL.

Unlock Jenkins using the initial admin password (shown in the command prompt).

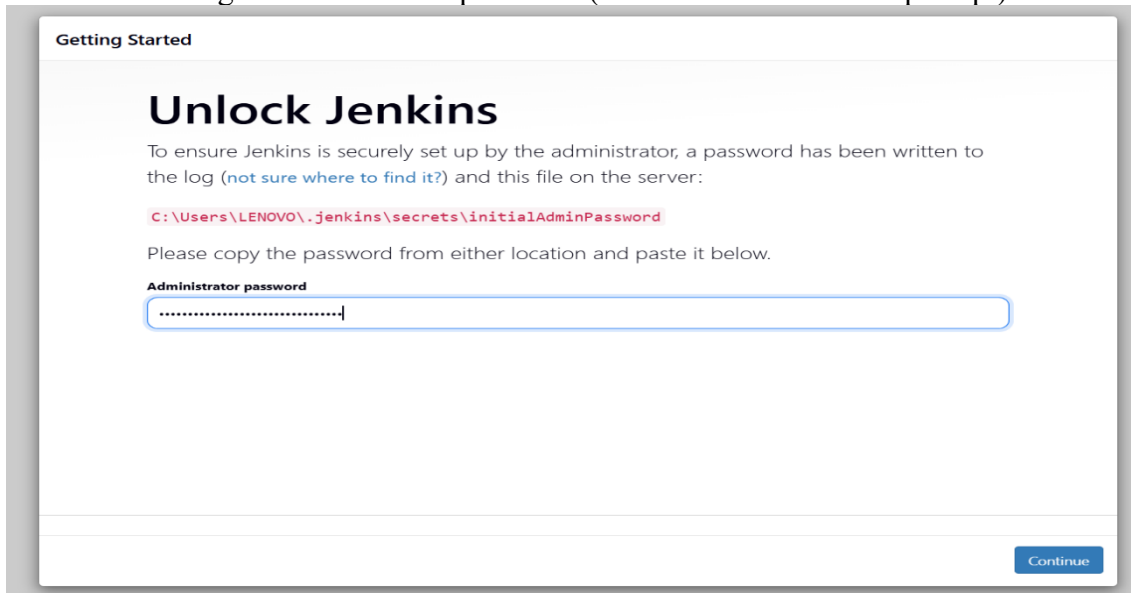


Fig 5.1: Unlocking Jenkins using initial admin password

Install suggested plugins and create an admin user.

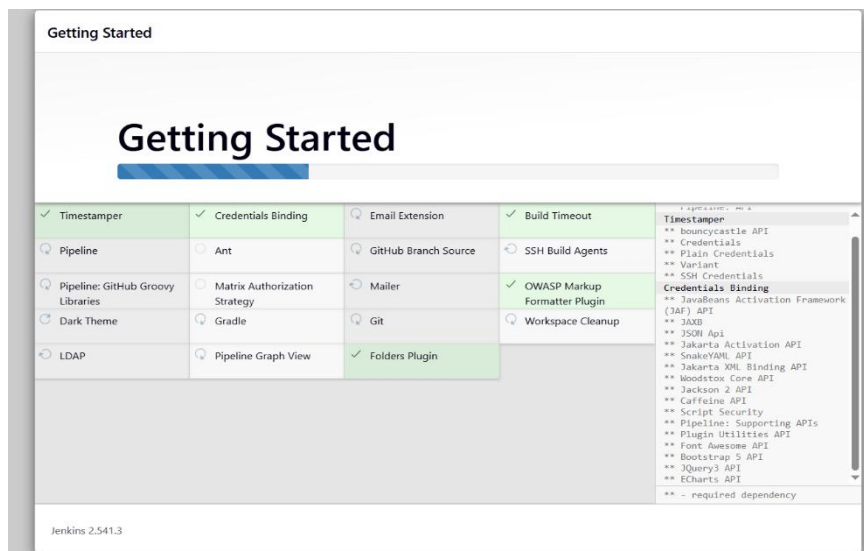
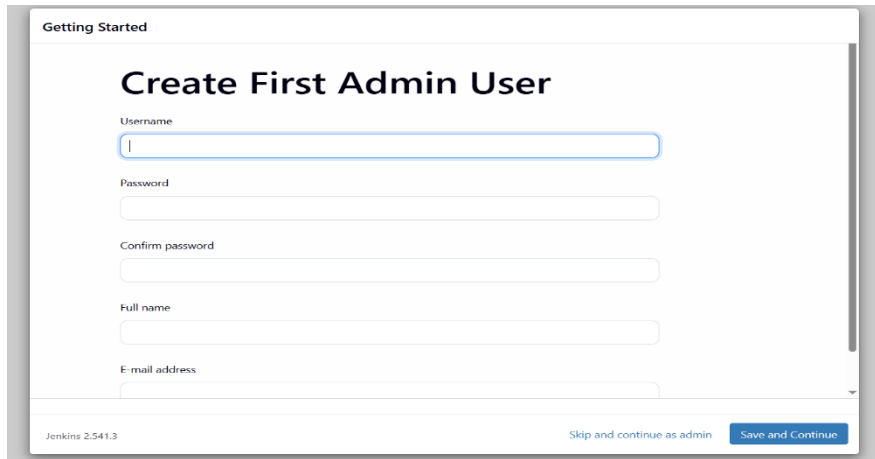


Fig 5.2: Installing suggested plugins and creating admin user

4. Create Admin User

- Fill in details:
 - Username
 - Password
 - Full Name
 - Email Address
- Click **Save and Continue**

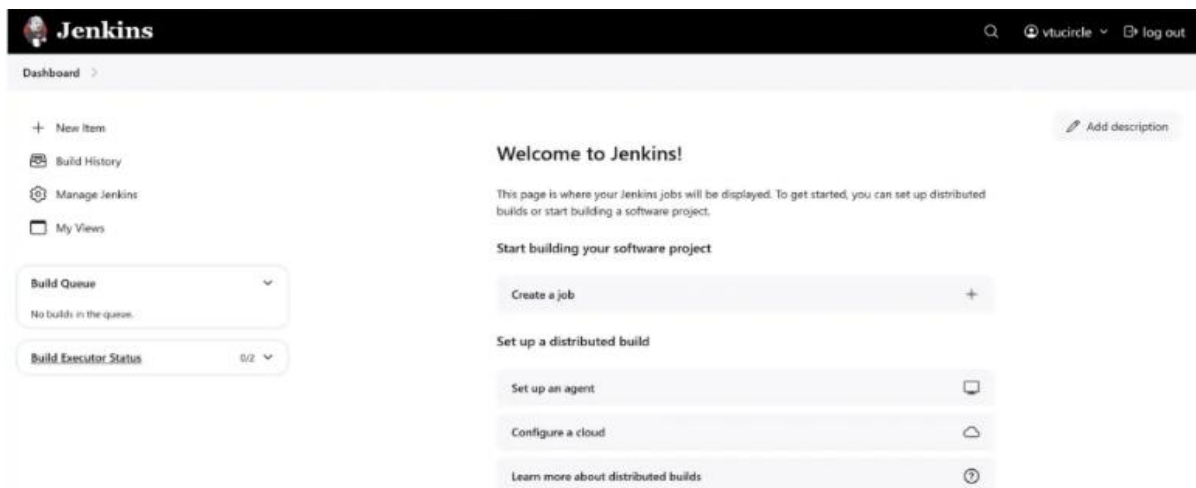


The image shows the 'Getting Started' screen of Jenkins with the title 'Create First Admin User'. It contains five input fields: 'Username', 'Password', 'Confirm password', 'Full name', and 'E-mail address'. At the bottom, there is a 'Skip and continue as admin' link and a 'Save and Continue' button. The version 'Jenkins 2.541.3' is displayed in the bottom left corner.

Fig 5.3: Jenkins Setup - Save and Continue

5.Jenkins setup in browse:

Fig 5.4: Jenkins Dashboard after initial setup



After saving the admin details, click on **Save and Finish** to complete the setup. Then click on **Start using Jenkins**, and you will be redirected to the Jenkins Dashboard where you can begin using Jenkins.

EXPERIMENT 6

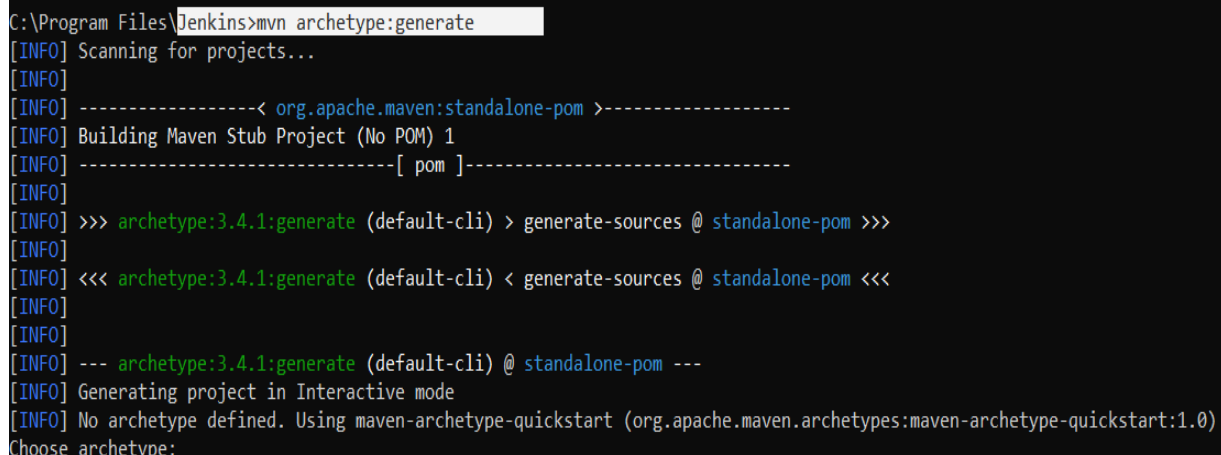
Continuous Integration with Jenkins: Setting Up a CI Pipeline, Integrating Jenkins with Maven/Gradle, Running Automated Builds and Tests.

Install and Start Jenkins

- Ensure Jenkins is installed and running:
java -jar jenkins.war --httpPort=8080
- Open in browser:
http://localhost:8080

Steps to create a new Maven Project:

1. Open Command Prompt. Give the command `mvn archetype:generate`

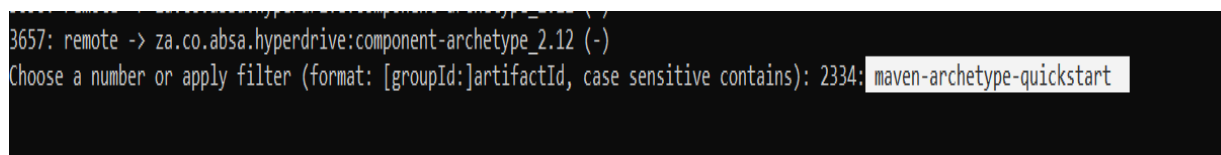


```
C:\Program Files\Jenkins>mvn archetype:generate
[INFO] Scanning for projects...
[INFO] -----< org.apache.maven:standalone-pom >-----
[INFO] Building Maven Stub Project (No POM) 1
[INFO] -----[ pom ]-----
[INFO] >>> archetype:3.4.1:generate (default-cli) > generate-sources @ standalone-pom >>>
[INFO] <<< archetype:3.4.1:generate (default-cli) < generate-sources @ standalone-pom <<<
[INFO] --- archetype:3.4.1:generate (default-cli) @ standalone-pom ---
[INFO] Generating project in Interactive mode
[INFO] No archetype defined. Using maven-archetype-quickstart (org.apache.maven.archetypes:maven-archetype-quickstart:1.0)
Choose archetype:
```

Fig 6.1: Creating Maven project using Command Prompt

2. Just type:

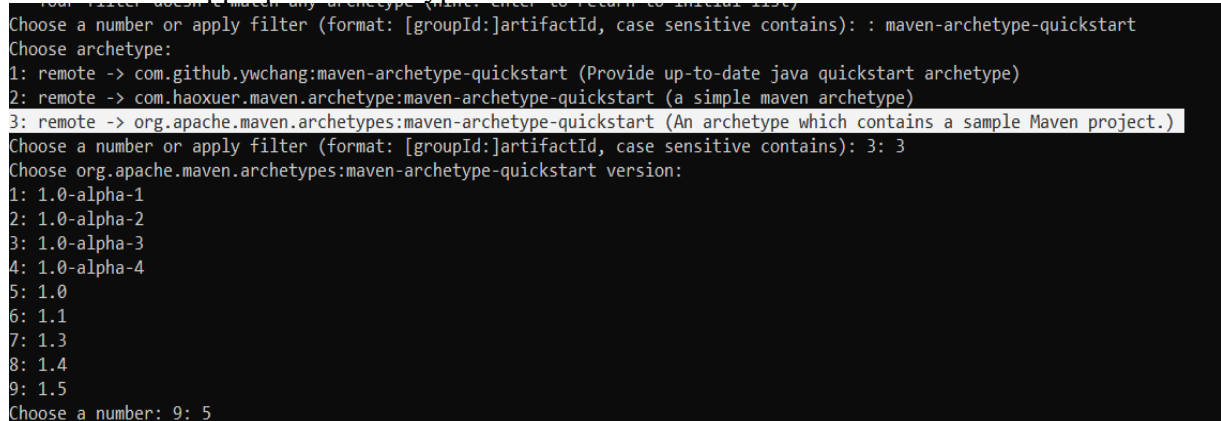
`maven-archetype-quickstart` Then press **Enter**. This is the standard basic Java project template.



```
3657: remote -> za.co.absa.hyperdrive:component-archetype_2.12 (-)
Choose a number or apply filter (format: [groupId:]artifactId, case sensitive contains): 2334: maven-archetype-quickstart
```

Fig 6.2: Standard basic Java project template selection

3. Choose the appropriate archetype and version.



```
Choose a number or apply filter (format: [groupId:]artifactId, case sensitive contains): : maven-archetype-quickstart
Choose archetype:
1: remote -> com.github.ywchang:maven-archetype-quickstart (Provide up-to-date java quickstart archetype)
2: remote -> com.haoxuer.maven.archetype:maven-archetype-quickstart (a simple maven archetype)
3: remote -> org.apache.maven.archetypes:maven-archetype-quickstart (An archetype which contains a sample Maven project.)
Choose a number or apply filter (format: [groupId:]artifactId, case sensitive contains): 3: 3
Choose org.apache.maven.archetypes:maven-archetype-quickstart version:
1: 1.0-alpha-1
2: 1.0-alpha-2
3: 1.0-alpha-3
4: 1.0-alpha-4
5: 1.0
6: 1.1
7: 1.3
8: 1.4
9: 1.5
Choose a number: 9: 5
```

Fig 6.3: Choosing the appropriate archetype and version

4. Enter the required project details:

groupId:org.example

artifactId:Program5

version: (press Enter)

package: (press Enter)

Then confirm by typing: Y

Final project structure created after success:

Program5/

```
|
├── pom.xml
└── src/
    ├── main/java/org/example/App.java
    └── test/java/org/example/AppTest.java
```

Modify pom.xml:

Open the pom.xml file and replace its content with the required configuration including dependencies for JUnit and TestNG, and the Maven Surefire Plugin for running tests.

```
<project xmlns="http://maven.apache.org/POM/4.0.0">

  <modelVersion>4.0.0</modelVersion>

  <groupId>org.example</groupId>
  <artifactId>demo</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
  </properties>

  <dependencies>

    <!-- Junit 5 (WORKING) -->
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter</artifactId>
      <version>5.10.2</version>
      <scope>test</scope>
    </dependency>
    <dependency>
      <groupId>org.testng</groupId>
      <artifactId>testng</artifactId>
      <version>RELEASE</version>
      <scope>test</scope>
    </dependency>
    <dependency>
      <groupId>junit</groupId>
```

```
<artifactId>junit</artifactId>
<version>4.13.2</version>
<scope>test</scope>
</dependency>

</dependencies>

<build>
  <plugins>

    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-surefire-plugin</artifactId>
      <version>3.2.5</version>
    </plugin>

  </plugins>
</build>

</project>
```

Update Test File

Open: [src/test/java/org/example/AppTest.java](#) Replace the existing code with your required test cases.

A screenshot of a code editor showing the file 'AppTest.java'. The code is as follows:

```
1 package org.example;
2
3 import org.junit.jupiter.api.Test;
4 import static org.junit.jupiter.api.Assertions.*;
5
6 public class AppTest {
7
8     @Test
9     void test1() { assertEquals( expected: 4, actual: 2 + 2); }
10
11     @Test
12     void test2() { assertEquals( expected: 10, actual: 5 + 5); }
13
14     @Test
15     void test3() { assertEquals( expected: 3, actual: 5 - 2); }
16
17     @Test
18     void test4() { assertEquals( expected: 12, actual: 3 * 4); }
19
20     @Test
21     void test5() { assertEquals( expected: 2, actual: 10 / 5); }
22 }
```

Fig 6.4: Replacing test case code in AppTest.java

To Run Project Locally Execute the following command in the terminal: mvn clean install

Configure Tools in Jenkins

1. Go to Manage Jenkins > Global Tool Configuration.
2. Under JDK, click Add JDK
 - o Name: JDK-17

- JAVA_HOME: C:\Program Files\Java\jdk-17
- 3. Under Maven, click Add Maven:
 - Name: Maven-3.9.12
 - Select version as 3.9.12

Install Required Plugins

1. Go to Manage Jenkins > Manage Plugins.
2. Under the Available tab, search for and install the following plugins:
 - Maven Integration Plugin

Step 1: Create a New Jenkins Job

1. Log into Jenkins
2. Create a New Job:
 - On the Jenkins dashboard, click on “New Item”
 - Enter an Item Name
 - select “Freestyle project”
 - Click “OK”.

New Item

Enter an item name

Exp 5

Select an item type

- Pipeline**
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
- Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
- Maven project**
Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.
- Multi-configuration project**
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
- Folder**
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.

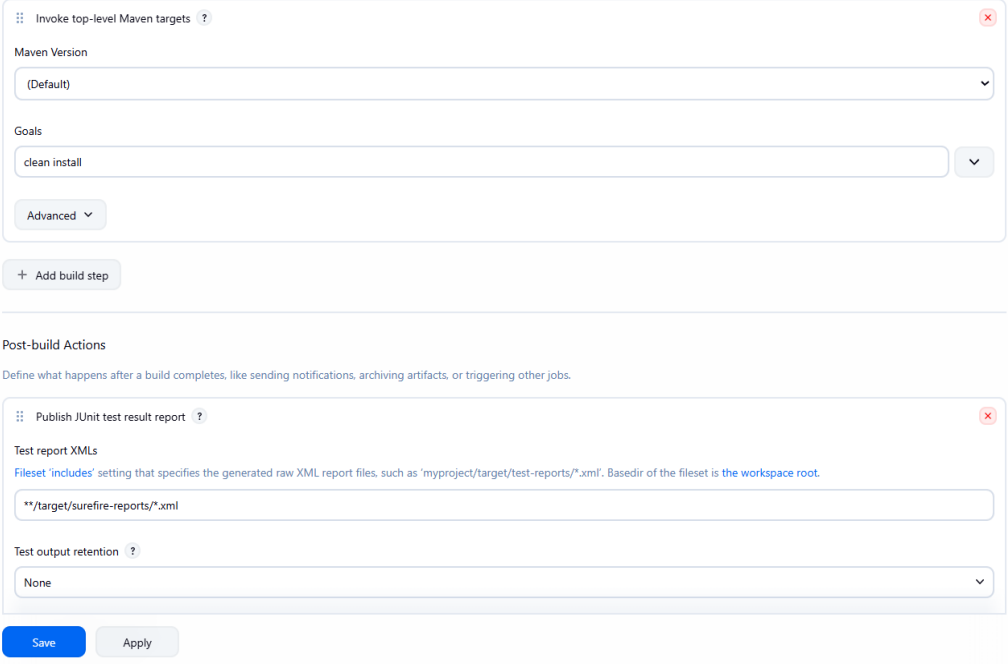
OK

Fig 6.5: Creating a Freestyle project in Jenkins

Step 2: Add Build Steps

- In the **Build** section, add **Invoke top-level Maven targets**.
- Set Maven version to *Default* and enter goals: clean install.
- In **Post-build Actions**, add **Publish JUnit test result report**.
- Set test report path to `**/target/surefire-reports/*.xml`.
- Save the job and click **Build Now** to run and view results.

Note: Ensure the Jenkins workspace path is correctly set. Example:
C:\Users\LENOVO\.jenkins\workspace\Project_Name



The screenshot displays the Jenkins configuration interface. At the top, there is a section titled 'Invoke top-level Maven targets' with a help icon. It contains a 'Maven Version' dropdown set to '(Default)', a 'Goals' dropdown set to 'clean install', and an 'Advanced' dropdown. Below this is a '+ Add build step' button. The next section is 'Post-build Actions', with a description: 'Define what happens after a build completes, like sending notifications, archiving artifacts, or triggering other jobs.' It contains a 'Publish JUnit test result report' step with a help icon. This step has a 'Test report XMLs' field with a fileset 'includes' setting, showing the path '**/target/surefire-reports/*.xml'. Below this is a 'Test output retention' dropdown set to 'None'. At the bottom of the configuration area are 'Save' and 'Apply' buttons.

Fig 6.6: Jenkins workspace directory path

Step 3: Click Build Now.

Step 4: After execution:

- Check **Console Output** for build logs.
- View **Test Results** section for JUnit reports

EXPERIMENT 7

Configuration Management with Ansible: Basics of Ansible: Inventory, Playbooks, and Modules, Automating Server Configurations with Playbooks, Hands-On: Writing and Running a Basic Playbook

1. What is Configuration Management?

Configuration Management (CM) in DevOps is the automated process of maintaining IT systems, software, and infrastructure in a known, consistent, and desired state. It eliminates the need for manual setup and tracking, which is essential for scaling modern, complex environments.

It ensures that:

- Systems are set up the same way every time
- Changes are tracked and controlled
- Errors due to manual setup are minimized

Key Objectives

- Consistency across multiple systems
- Automation of setup and updates
- Version control of configurations
- Faster deployment and recovery

Configuration management tools like Ansible, Puppet, Chef, and SaltStack are specifically designed to:

- Install software
- Maintain system state
- Ensure consistency across servers

2. Introduction to Ansible

Ansible is an open-source automation tool used for managing IT infrastructure efficiently. It allows users to automate repetitive tasks and manage multiple systems simultaneously.

Key Capabilities

- Configuration management
- Application deployment
- Infrastructure provisioning

Ansible is written in Python, Shell, and PowerShell. It was originally developed by Michael DeHaan and later acquired by Red Hat in 2015.

3. Ansible Workflow

Ansible operates using a simple workflow where a central system (controller) manages multiple machines (hosts) using defined instructions.

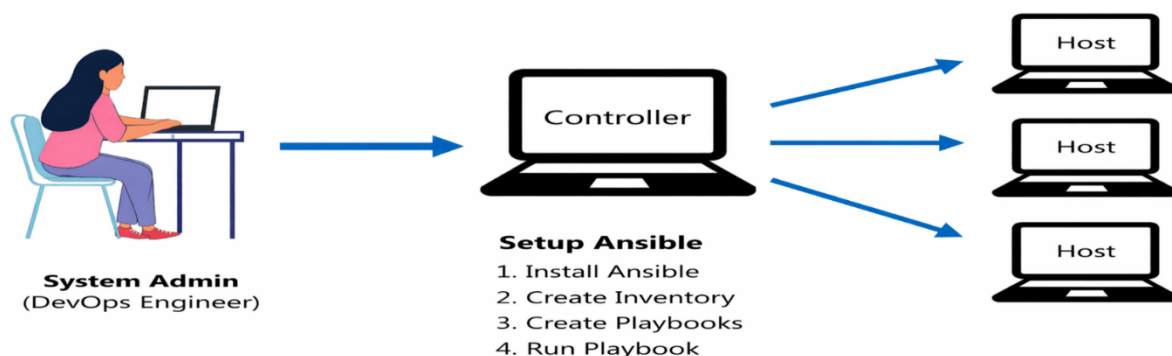


Fig 7.1: Ansible Workflow Overview

4. Ansible Architecture

Ansible follows a simple push-based architecture:

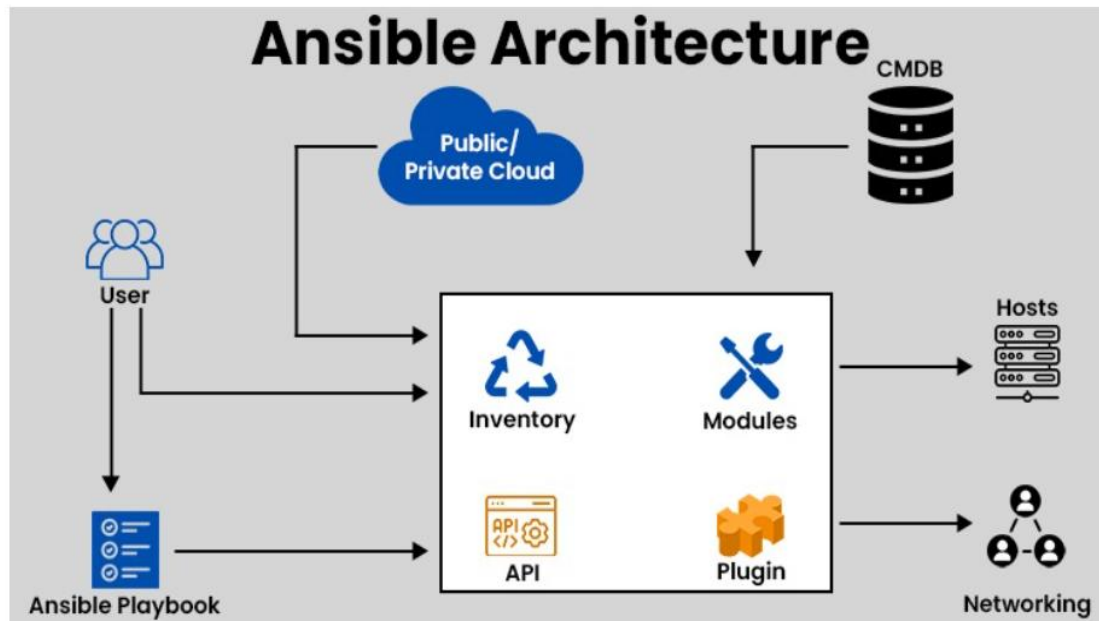


Fig 7.2: Ansible Push-based Architecture

Components:

- Control Node → Machine where Ansible is installed
- Managed Nodes → Target systems (servers)
- Inventory → List of servers
- Playbooks → Instructions (YAML)
- Modules → Tasks executed on nodes

5. Inventory and Playbooks

Inventory

- A file that contains details about the systems (hosts) to be managed
- Includes:
 - IP addresses or domain names
 - Group classifications (e.g., web servers, database servers)
 - Connection details (username, password, SSH info)

Playbooks

- Written in YAML format
- Contain instructions/tasks to be executed on hosts
- Define what actions should be performed and on which systems

Key Difference

- Inventory → Stores host data
- Playbook → Stores execution instructions

6. Controller (Control Node) and Host (Managed Node)

Controller	Host
Machine where Ansible is installed and commands are executed	Machines managed by Ansible
Manages configuration and deployment	Executes tasks sent by controller
Can be a laptop, desktop, VM, or cloud instance	Can be physical, virtual, or cloud-based systems

7. How Ansible Works

- The controller connects to hosts using SSH (Secure Shell)
- No agent installation is required on hosts
- Steps:
 1. Define hosts in the inventory file
 2. Write playbooks with required tasks
 3. Run playbooks from the controller
 4. Tasks execute on target hosts

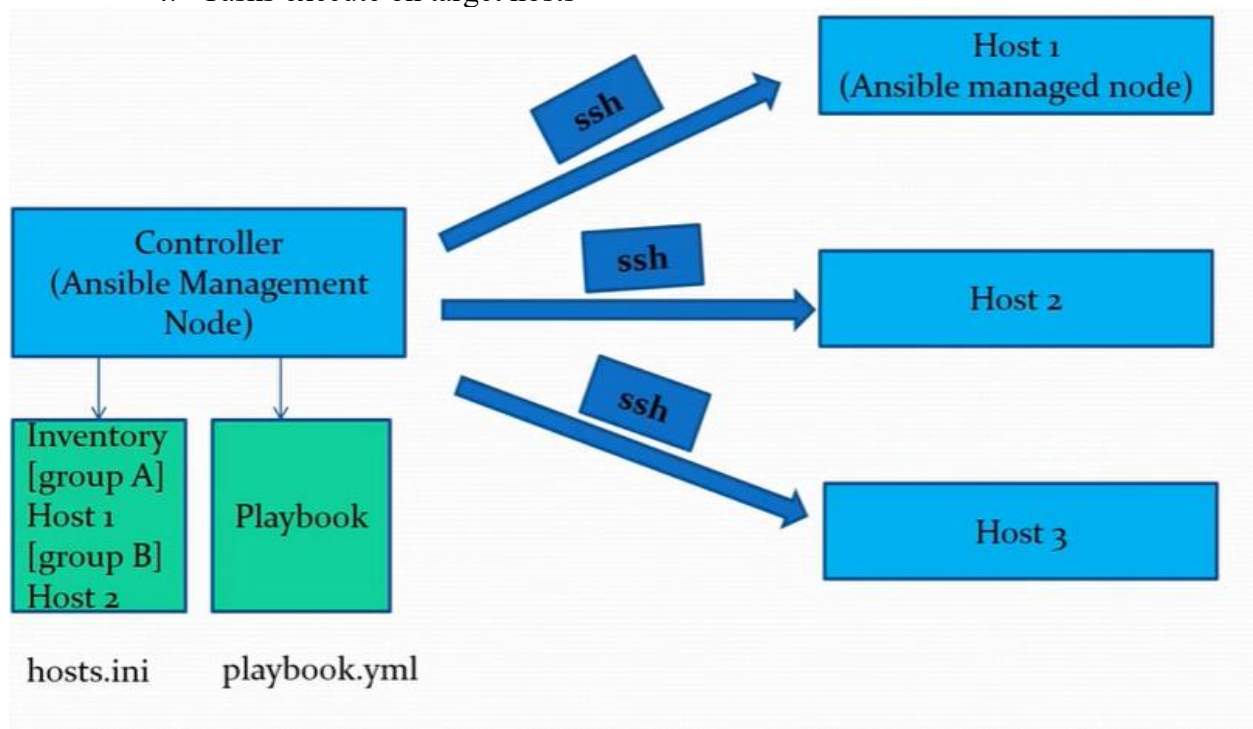


Fig 7.3: Tasks executing on target hosts

7. Example Setup

Use an Ubuntu system on VirtualBox as the controller

Step 1: Install Ansible on Ubuntu

```
sudo apt update  
sudo apt install ansible -y
```

Step 2: Check Ansible is Installed

```
ansible -version
```

Fig 7.4: Example Setup using Ubuntu on VirtualBox

Step 3: Test Connection with localhost - Create or edit the Ansible inventory file

```
sudo nano /etc/ansible/hosts
```

Add:

```
[local]  
localhost ansible_connection=local
```

Fig 7.5: Ansible Controller Configuration

Then test:

```
ansible localhost -m ping
```

(You should see a “**pong**” response indicating success)

```
[root@centos01 ~]# ansible localhost -m ping  
localhost | SUCCESS => {  
  "changed": false,  
  "ping": "pong"
```

Fig 7.6: Ansible Inventory Setup

Step 4: Create a Directory for Playbooks

```
mkdir ~/ansible-playbooks  
cd ~/ansible-playbooks
```

Fig 7.7: Ansible Playbook structure

Step 5: Create First Playbook File - Create the file firstpb.yml

```
nano firstpb.yml
```

Code:

```
---
- name: First Basic PB
  hosts: localhost

  tasks:
    - name: Test Connectivity
      ping:
    - name: "Print Output"
      debug: msg="Alright!!"
```

After writing the code, save the file and exit the editor.

Fig 7.8: Ansible module execution

Step 6: Run the Playbook

```
ansible-playbook firstpb.yml
```

Fig 7.9: Ansible task output

You should see both:

- ✓ ping success message
- ✓ debug output: "Alright!!"

```
ok: [localhost]

TASK [Test Connectivity] ***
*****
ok: [localhost]

TASK [Print Output] *****
*****
I
ok: [localhost] => {
  "msg": "Alright!!"
}
```

Fig 7.10: Ansible playbook run result

EXPERIMENT 8

Practical Exercise: Set Up a Jenkins CI Pipeline for a Maven Project, Use Ansible to Deploy Artifacts Generated by Jenkins

Step 1: Create a simple Maven 'HelloWorld' project using IntelliJ IDEA and Run a test build.

Step 2: Create a Jenkins Job

- **Go to Jenkins Dashboard** → Click **New Item** → **Freestyle Project**
- **Enter Job Name:** HelloWorld-Maven-Build
- **Select Git** as Source Code Management

• **Repository URL:** Use a local Git repo or push your Maven project to GitHub and use that link.

Step 3: Configure Build

- Under **Build**, add:

```
mvn clean package
```

Fig 8.1: Jenkins Build Configuration - Add Execute Shell

- Under Post-build Actions, select:
 - Archive the artifact:

```
target/helloworld-1.0-SNAPSHOT.jar
```

Fig 8.2: Jenkins Post-build Actions Configuration

Save and Build the Project.

Step 4: Create an Ansible Playbook for Deployment

Create the directory for the playbook:

```
mkdir -p ~/ansible-deploy  
cd ~/ansible-deploy  
nano deploy.yml
```

Fig 8.3: Creating directory for Ansible playbook

Add the following Ansible playbook:

```
---
- name: Deploy HelloWorld App hosts:
  local
  tasks:
    - name: Copy the JAR file to the deployment directory copy:
      src: /var/lib/jenkins/workspace/HelloWorld-Maven-
Build/target/helloworld-1.0-SNAPSHOT.jar dest:
      /opt/myapp/helloworld.jar mode: '0755'

    - name: Restart the application
      shell: "nohup java -jar /opt/myapp/helloworld.jar >
```

Fig 8.4: Adding Ansible playbook content

Save and exit.

Step 5: Integrate Ansible with Jenkins

- Open Jenkins Dashboard → Select **HelloWorld-Maven-Build** job
- Go to **Configure** → **Post-build Actions**
- Click "**Invoke Ansible Playbook**"
 - **Playbook Path:** /home/user/ansible-deploy/deploy.yml
 - **Inventory File:** /etc/ansible/hosts

Save and Trigger the Build.

EXPERIMENT 9

Introduction to Azure DevOps: Overview of Azure DevOps Services, Setting Up an Azure DevOps Account and Project.

Overview of Azure DevOps Services

1. What is Azure DevOps?

Azure DevOps is a cloud-based platform from Microsoft that provides a **complete set of tools to build, manage, test, and deploy software**.

Think of it as an **all-in-one toolkit for the entire software development lifecycle**—from planning a project to delivering it to users.

Azure DevOps helps teams:

- Plan work
- Write and store code
- Build and test applications
- Deploy software automatically

Core Services in Azure DevOps

Azure DevOps is made up of several integrated services:

1. Azure Boards

- Agile project management (Scrum, Kanban)
- Track tasks, bugs, and features
- Sprint planning and progress tracking

2. Azure Repos

- Git repositories for storing code
- Supports version control, branching, and code reviews

3. Azure Pipelines

- Automates build, test, and deployment
- Supports CI/CD (Continuous Integration & Delivery)
- Works with any language or platform

4. Azure Test Plans

- Manage and run tests (manual + automated)
- Track bugs and test results

5. Azure Artifacts

- Manage and share packages (npm, Maven, NuGet, etc.)
- All these tools work together to support the full development lifecycle.

Why Azure DevOps is Useful

- **End-to-end solution** → Everything in one platform
- **Collaboration** → Developers, testers, and managers work together
- **Automation** → Reduces manual work (builds, testing, deployment)
- **Scalable** → Works for small teams and large enterprises
- **Flexible** → Supports any language, platform, or cloud

In summary Azure DevOps = Tools + Automation + Collaboration for building and delivering software faster.

How Azure DevOps services work together?

The following diagram shows how the services integrate throughout the development lifecycle:

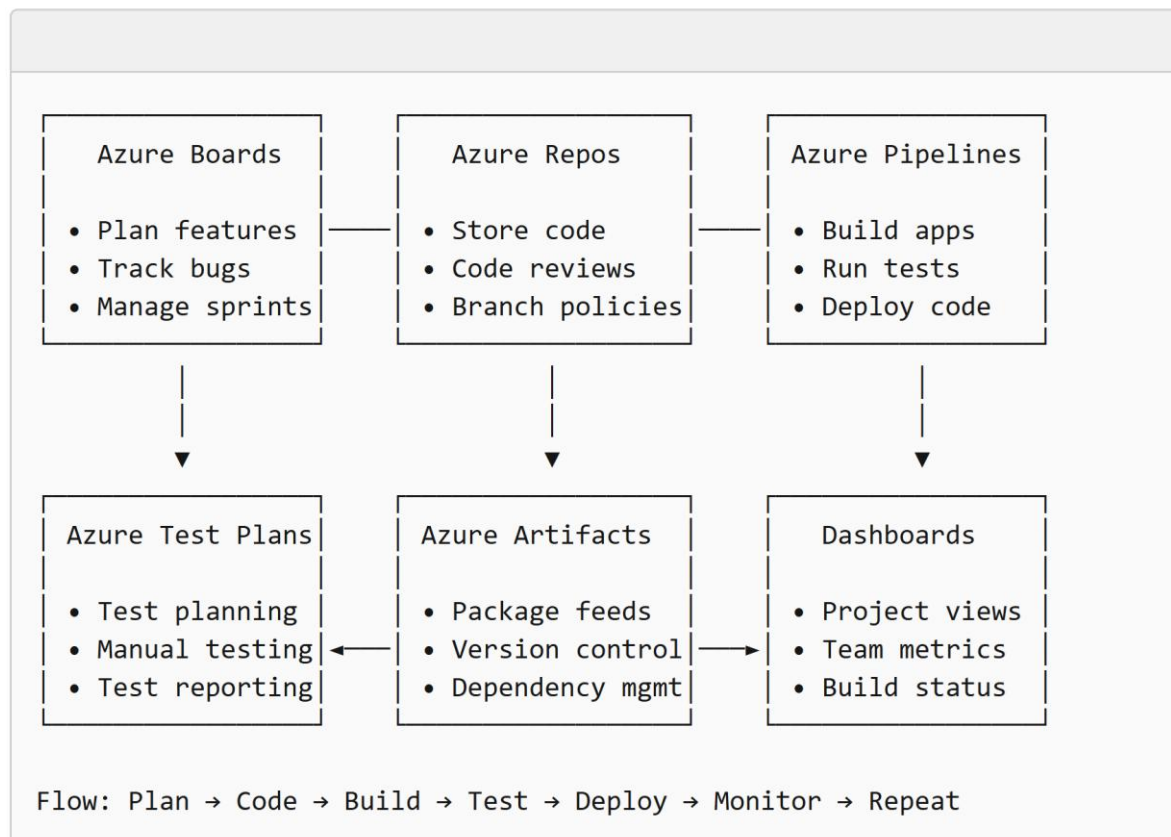


Fig 9.1: Azure DevOps Services Integration Lifecycle Diagram

Typical workflow:

1. **Plan** work items in Azure Boards
2. **Code** features in Azure Repos with pull requests
3. **Build** and package with Azure Pipelines and Azure Artifacts
4. **Test** manually and automatically using Azure Test Plans
5. **Deploy** through Azure Pipelines to various environments
6. **Monitor** progress and metrics via Dashboards
7. **Iterate** based on feedback and new requirements

Fig 9.2: Azure DevOps Portal

Setting Up an Azure DevOps Account and Project

Step1: visit “ <https://azure.microsoft.com/en-in/products/devops/> “and click on “**Get started with Azure**” button. User must login with existing Microsoft account or create a new Microsoft account to proceed.

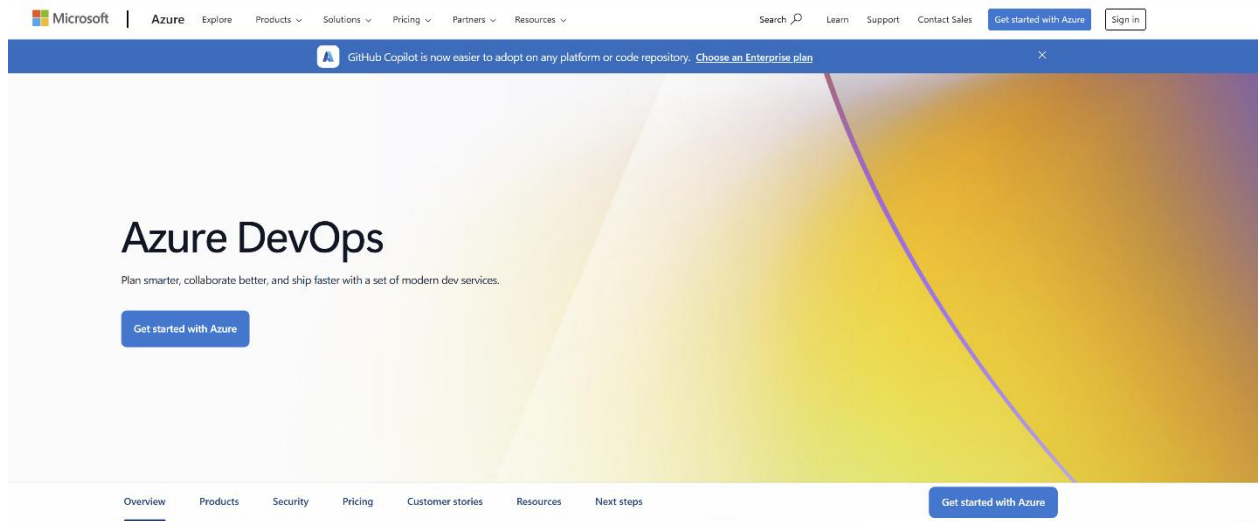


Fig 9.3: Azure DevOps - Get started with Azure

STEP 2: After logging in, search for "azure devops" in the top search bar and select "Azure DevOps organizations" from the list.

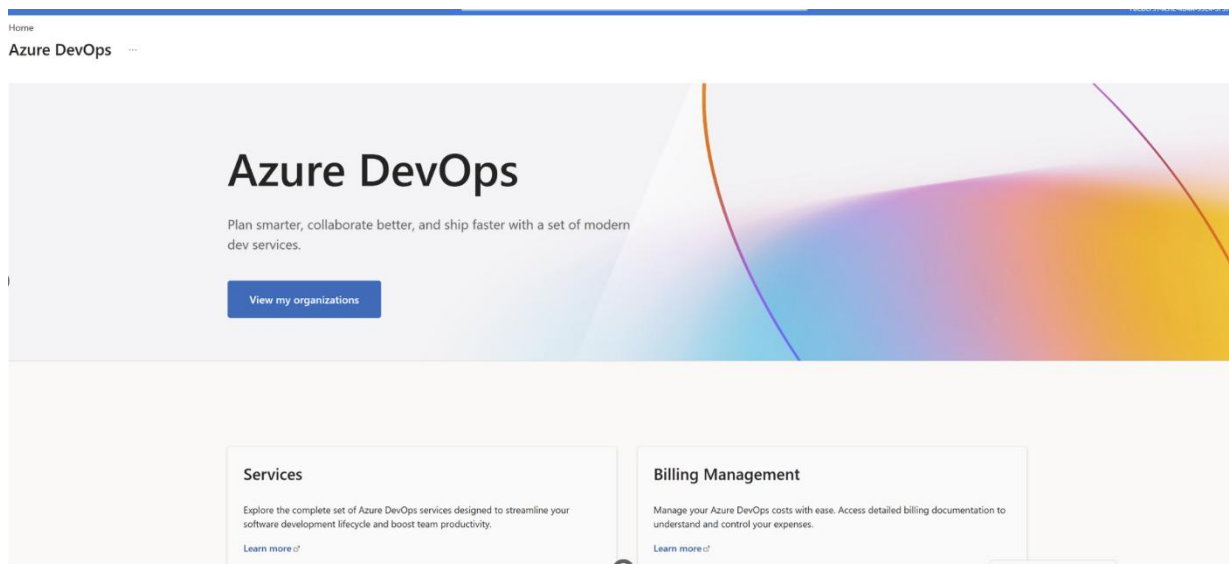
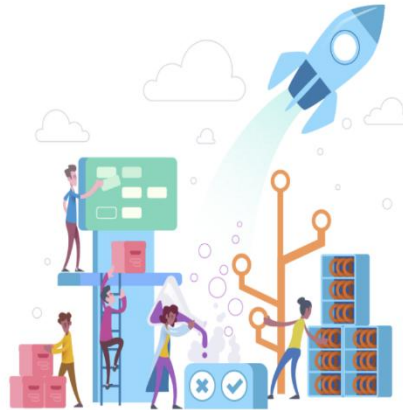


Fig 9.4: Searching Azure DevOps organizations in search bar

STEP 3: On the Azure DevOps page, click " View My Organizations" at the bottom left to view or create your DevOps organization.



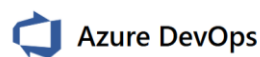
Get started with Azure DevOps

Plan better, code together, ship faster with Azure DevOps

Create new organization

Fig 9.5: Azure DevOps page - View My Organizations

STEP 4: After clicking "Create new Organizations", you will see your organization details, where you can click "Create new organization" to start creating a DevOps organization.



ashwinichidgallu@gmail.com [Switch directory](#)

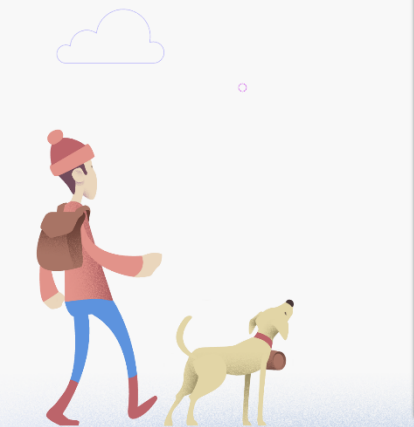
Get started with Azure DevOps

Choosing **Continue** means that you agree to our [Terms of Service](#), [Privacy Statement](#), and [Code of Conduct](#).

☐ I would like information, tips, and offers about Azure DevOps and other Microsoft products and services. [Privacy Statement](#).

Continue

Fig 9.6: Creating a new Azure DevOps Organization



ashwinichidgallu@gmail.com

Almost done...

Name your Azure DevOps organization *

dev.azure.com/ KVGCE-AIML

We'll host your projects in

India

Select an Azure subscription for billing [Link](#)

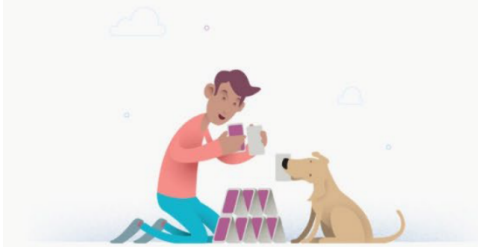
To create an Azure DevOps organization, you need to link it to an Azure subscription. We couldn't find any subscriptions you have access to. Make sure you have the Owner or Contributor role on at least one Azure subscription.

[Get started with Azure](#)

[Continue](#)

Fig 9.7: Organization details and Create new organization

STEP 7: Enter your project name, optional description, choose visibility (Private/Public), and click "Create project" to set up your first Azure DevOps project.



Search

Create a project to get started

Project name *

Demo

Description

Visibility

Public
Anyone on the internet can view the project. Certain features like TFVC are not supported.

Private
Only people you give access to will be able to view this project.

Public projects are disabled for your organization. You can turn on public visibility with [organization policies](#).

Advanced

[+ Create project](#)

Fig 9.8: Creating first Azure DevOps project

EXPERIMENT 10

Creating Build Pipelines: Building a Maven/Gradle Project with Azure Pipelines, Integrating Code Repositories (e.g., GitHub, Azure Repos), Running Unit Tests and Generating Reports.

Step 1: After logging into **Azure DevOps**, click your organization name (top-left) → go to **Organization Settings**. In the **Overview** section, verify the name (e.g., krishnagudi0948), region (e.g., Central India), and set the time zone if needed.

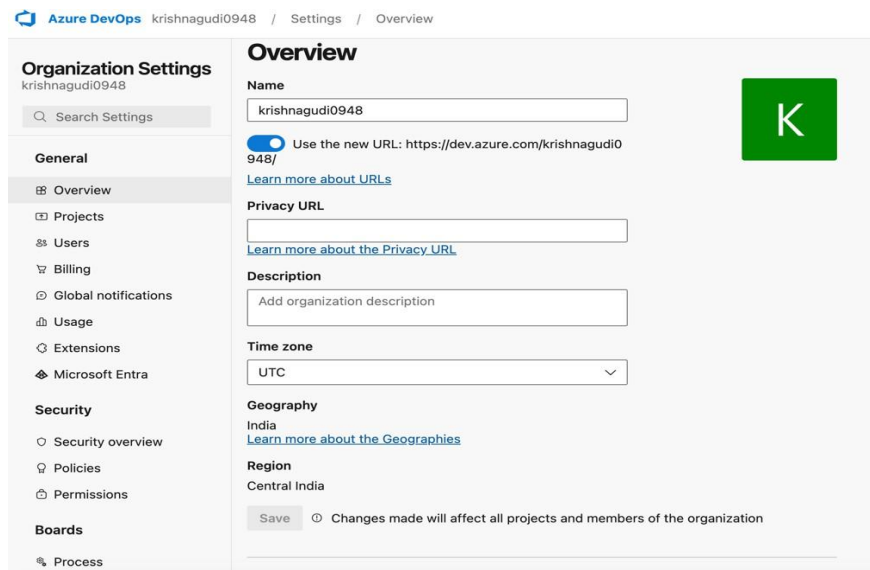


Fig 10.1: Azure DevOps Organization Settings Overview

Step 2: Navigate to **Organization Settings > Policies**.

Ensure that: **Public projects** are allowed (turn ON), **SSH Authentication** is ON, **Invite GitHub users** can remain OFF (optional).

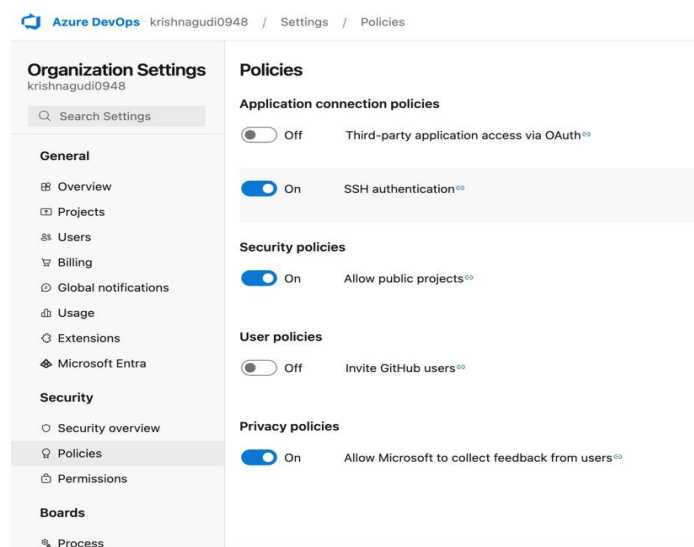


Fig 10.2: Organization Settings - Policies configuration

Step 3: From your Azure DevOps homepage, create a new project (e.g., demo1). This is where your pipeline and linked services will reside.

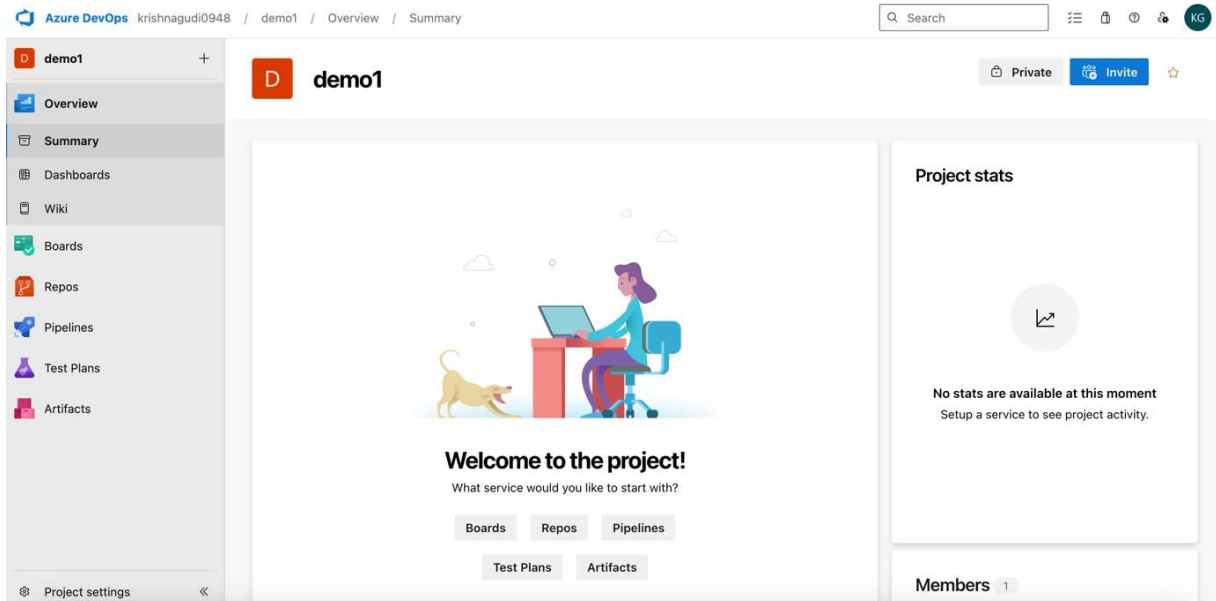


Fig 10.3: Creating a new project in Azure DevOps

Step 4: Inside the project, go to **Pipelines > Pipelines** and click **Create Pipeline** to start the CI/CD setup wizard.

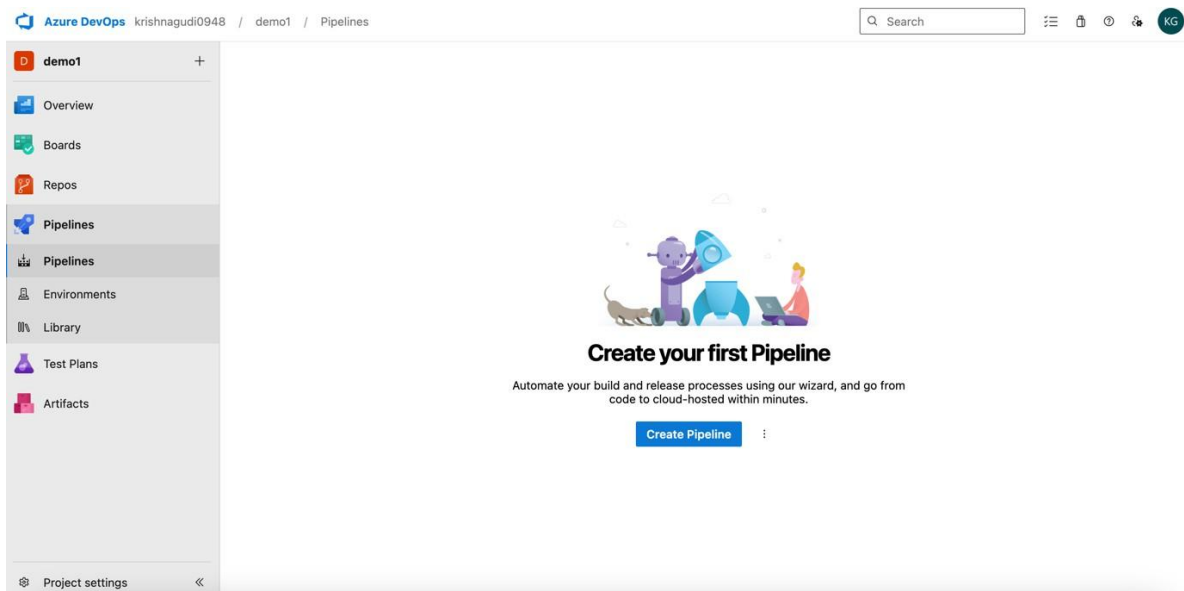


Fig 10.4: Pipelines - Create Pipeline wizard

Step 5: Connect to Your GitHub Repository: When prompted for your code source, select **GitHub**. You'll be redirected to authenticate and allow access.

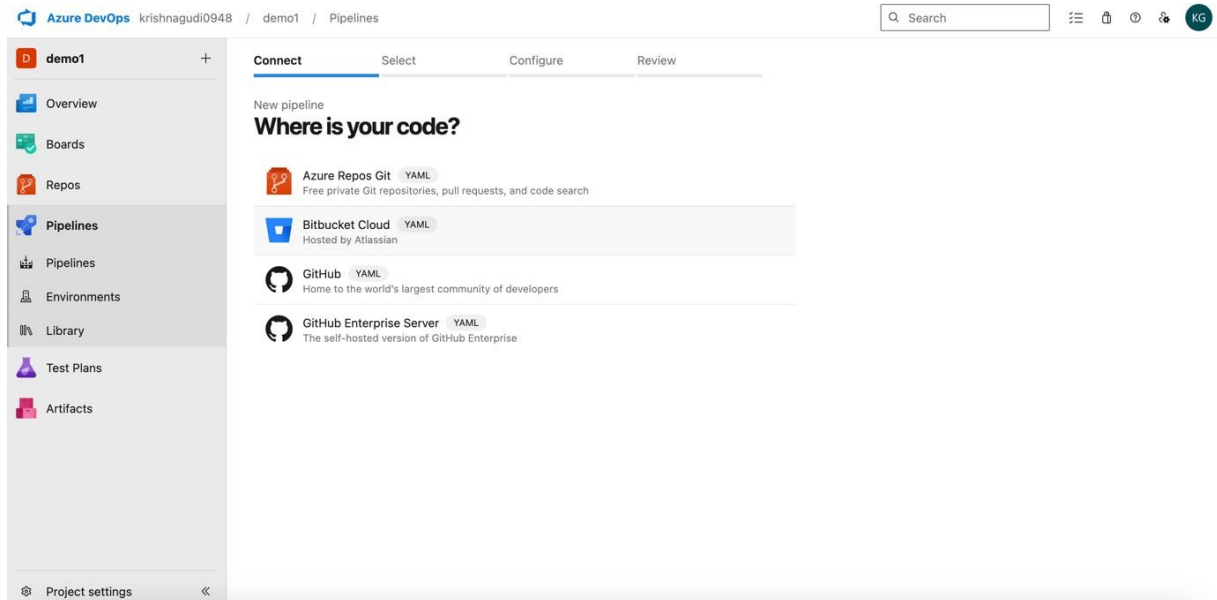


Fig 10.5: Connecting to GitHub as code source

Step 6: Choose Your Repository : Select the Maven project repository you want to build, for example: ksgudi/FirstProject.

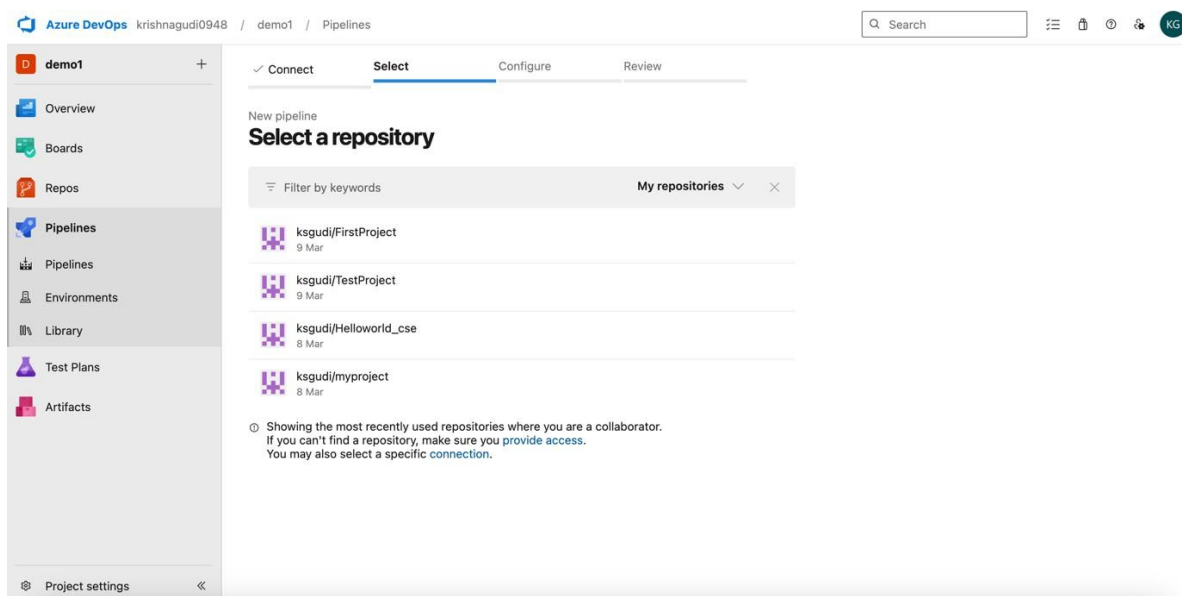


Fig 10.6: Choosing the GitHub repository

Step 7: Approve GitHub Integration: Approve access for Azure Pipelines to your GitHub repository. Choose “Only select repositories” and confirm access for FirstProject.

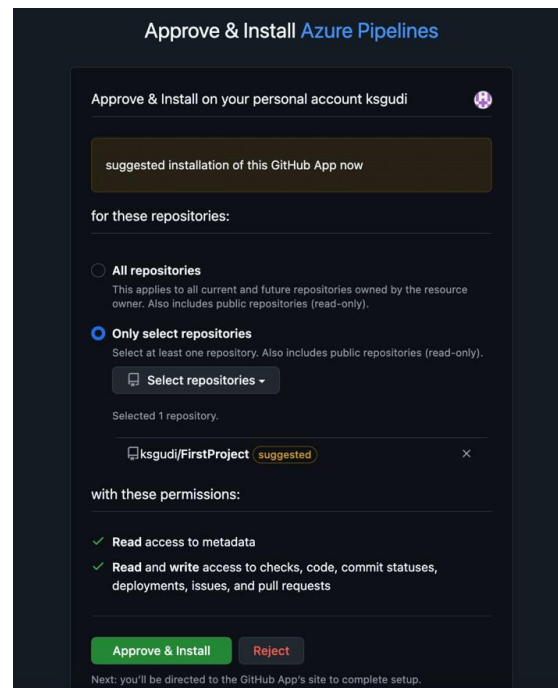


Fig 10.7: Approving Azure Pipelines GitHub integration

Step 8: Select Maven Pipeline Template: When prompted to configure the pipeline, choose **Maven** to auto-generate a pipeline template suitable for building and testing a Maven project.

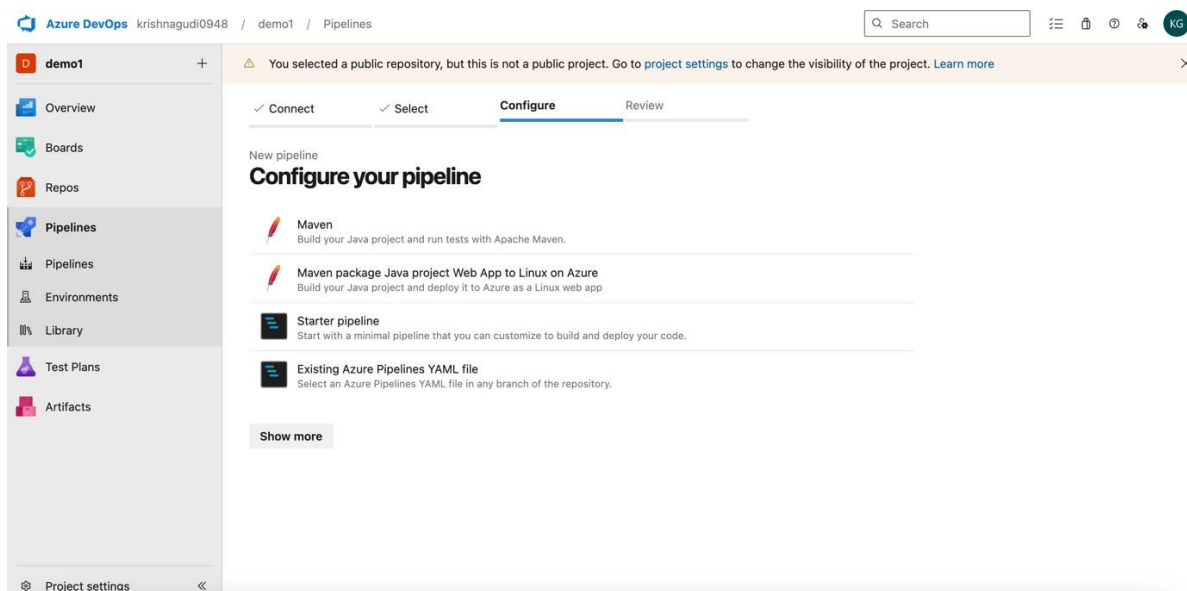


Fig 10.8: Selecting Maven Pipeline Template

Step 9: Review and Customize YAML: Review the auto-generated azure-pipelines.yml. Ensure it contains the Maven@3 task with appropriate goals like clean install and test result file paths.

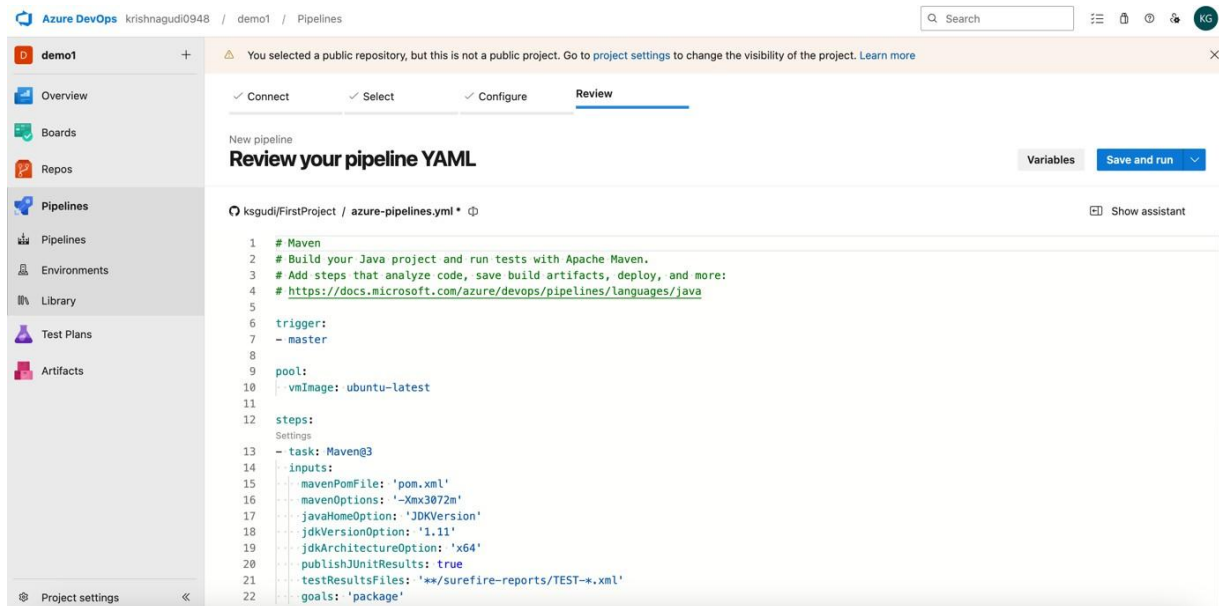


Fig 10.9: Review and Customize YAML with Maven@3 task

Step 10: Click Save and Run to trigger the pipeline.

⚠️ If the run fails with a message like "No hosted parallelism has been purchased," you must request free hosted agent access.

Fig 10.10: Save and Run to trigger the pipeline

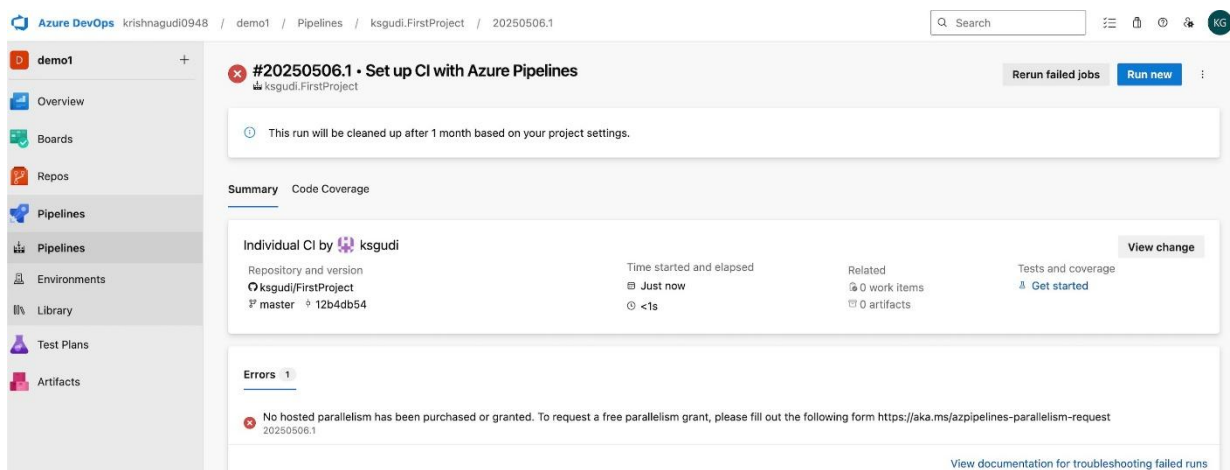


Fig 10.11: Pipeline run failed - No hosted parallelism purchased

Azure DevOps is **refusing to run your pipeline** because:

- Your organization **doesn't have permission** to use Microsoft-hosted agents (virtual machines that run your pipeline).
- By default, **new organizations created after certain limits** must **request access to free parallel jobs**, even for public projects.

Solution:

👉 **Request Free Parallelism**

Fig 10.12: Requesting Free Parallelism Solution

1. Go to this Microsoft form:
<https://aka.ms/azpipelines-parallelism-request>
2. Fill in the required details:

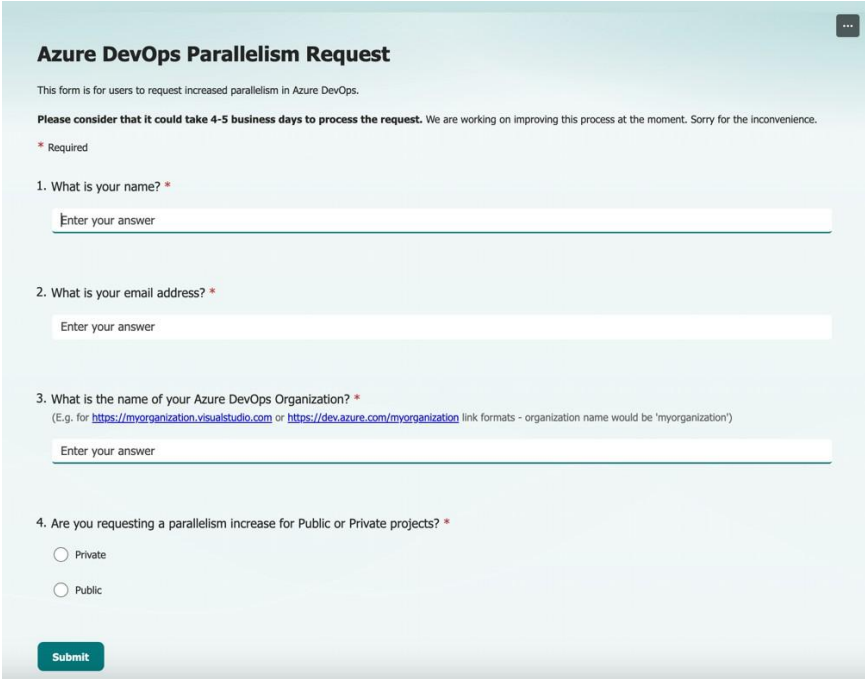
The image shows a web form titled "Azure DevOps Parallelism Request". The form is light blue with a white background. It contains the following elements: a title "Azure DevOps Parallelism Request", a subtitle "This form is for users to request increased parallelism in Azure DevOps.", a warning "Please consider that it could take 4-5 business days to process the request. We are working on improving this process at the moment. Sorry for the inconvenience.", a legend "* Required", and four numbered questions. Question 1: "What is your name?" with a text input field. Question 2: "What is your email address?" with a text input field. Question 3: "What is the name of your Azure DevOps Organization?" with a text input field and a note "(E.g. for https://myorganization.visualstudio.com or https://dev.azure.com/myorganization link formats - organization name would be 'myorganization')". Question 4: "Are you requesting a parallelism increase for Public or Private projects?" with two radio buttons labeled "Private" and "Public". A green "Submit" button is at the bottom left.

Fig 10.13: Filling Free Parallelism Request Form

1. Wait for approval (usually within a few hours to 2 days).

EXPERIMENT 11

Creating Release Pipelines: Deploying Applications to Azure App Services, Managing Secrets and Configuration with Azure Key Vault, Hands-On: Continuous Deployment with Azure Pipelines.

Step 1: Ensure all recommended settings are enabled under Organization Settings to enforce secure and modern pipeline practices.

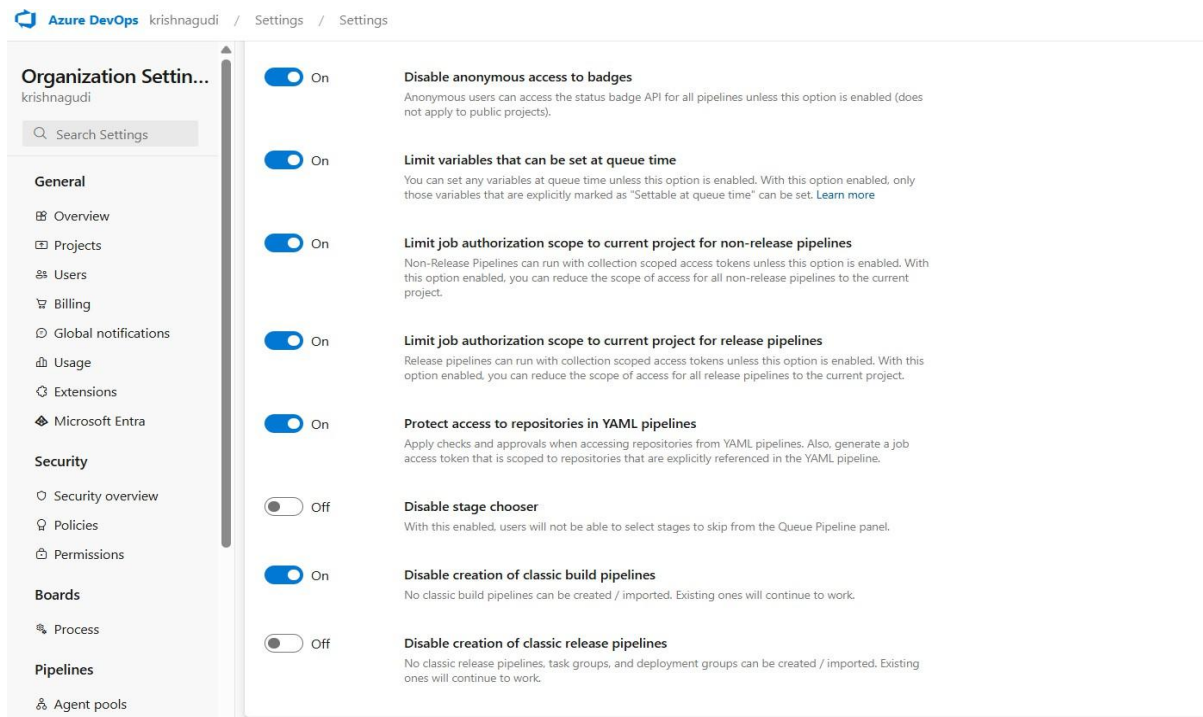


Fig 11.1: Organization Settings - Recommended settings enabled

Step 2: After build process, the run failed as no parallelism is granted—students must request free parallelism using the given form.

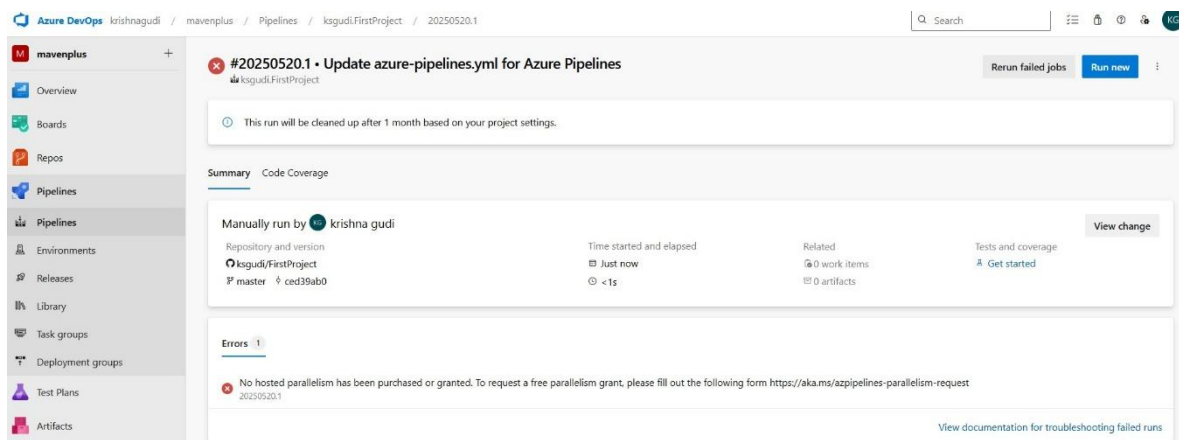


Fig 11.2: Build process run failed - No parallelism granted

Step 3: No release pipeline exists—click on ‘New pipeline’ to start creating one for deployment.

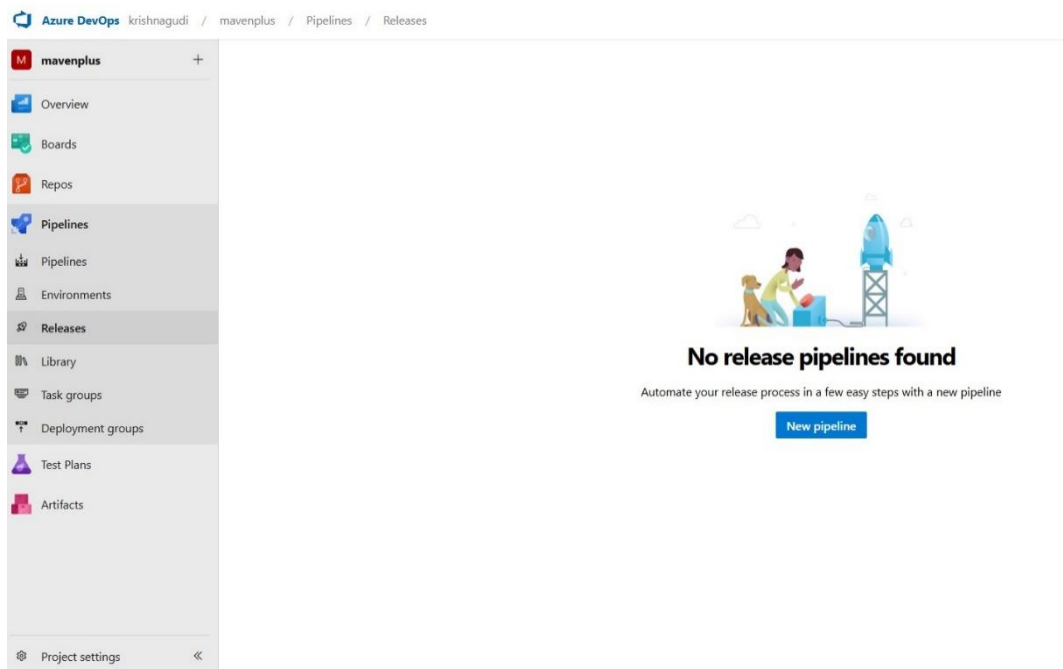


Fig 11.3: No release pipeline exists - New pipeline creation

Step 4: Begin your release pipeline by adding an artifact and at least one stage to proceed.

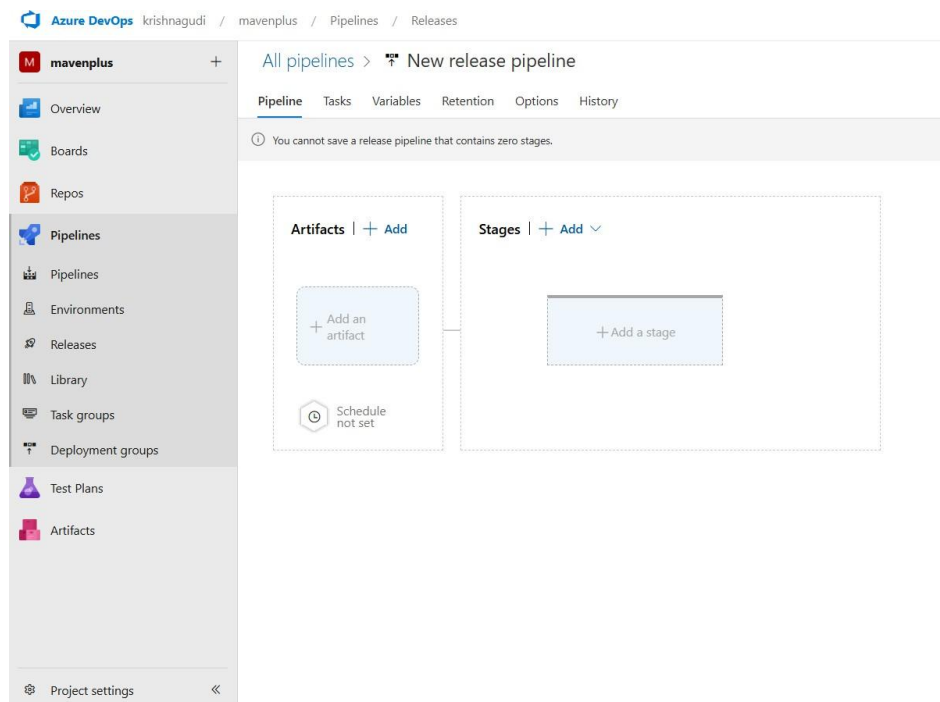


Fig 11.4: Adding artifact and stage to release pipeline

Step 5: Select the build pipeline as the artifact source to link your build output to the release pipeline.

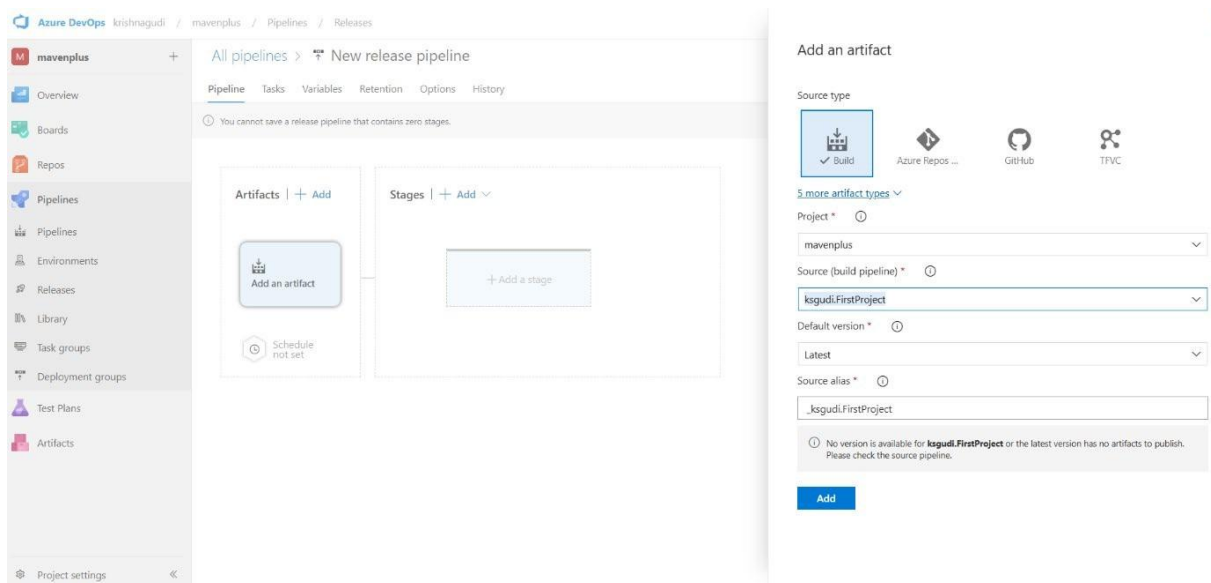


Fig 11.5: Selecting build pipeline as artifact source

Step 6: After adding the artifact, add a stage and choose a deployment template or start with an empty job.

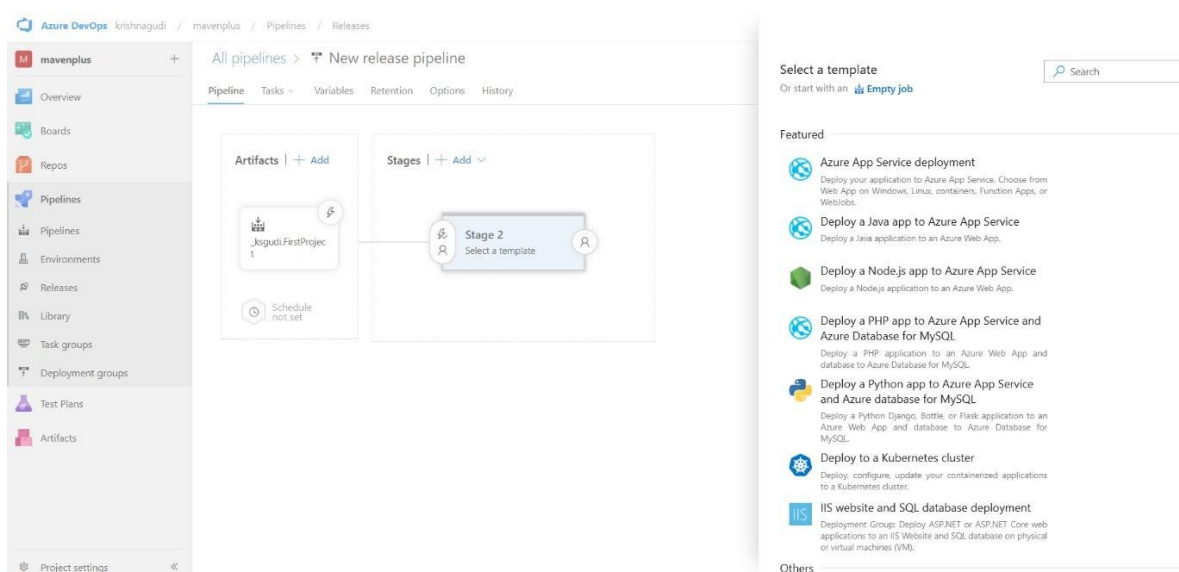


Fig 11.6: Adding a stage with deployment template

Step 7: Stage is created—click to configure jobs and tasks for deployment.

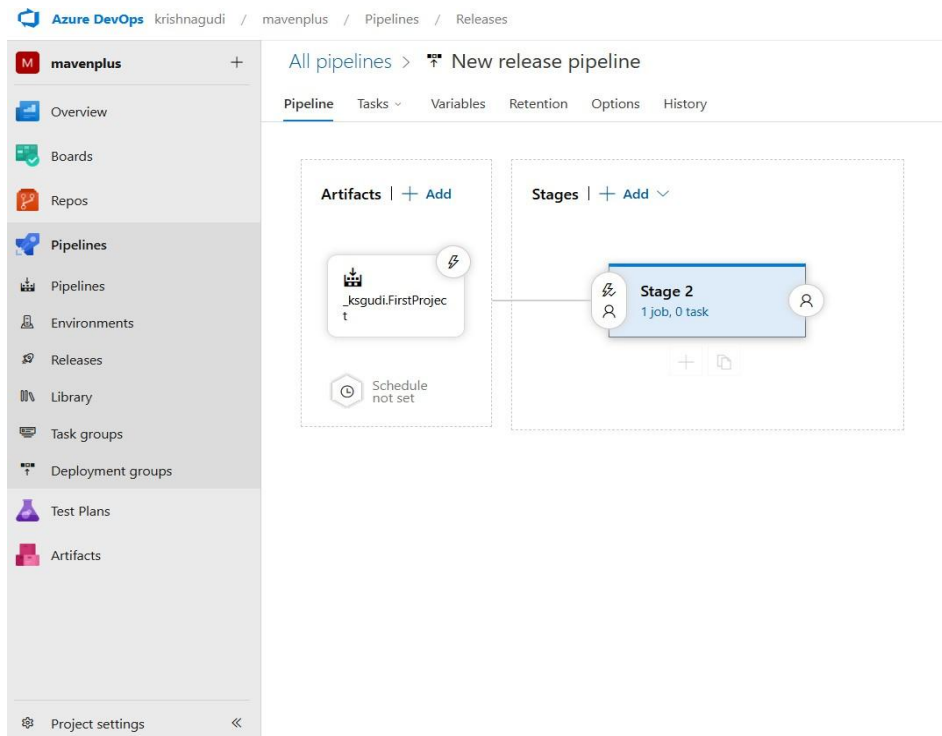


Fig 11.7: Stage created - Configure jobs and tasks

Step 8: Under 'Tasks', add a new deployment task such as a command line script to simulate or execute deployment steps.

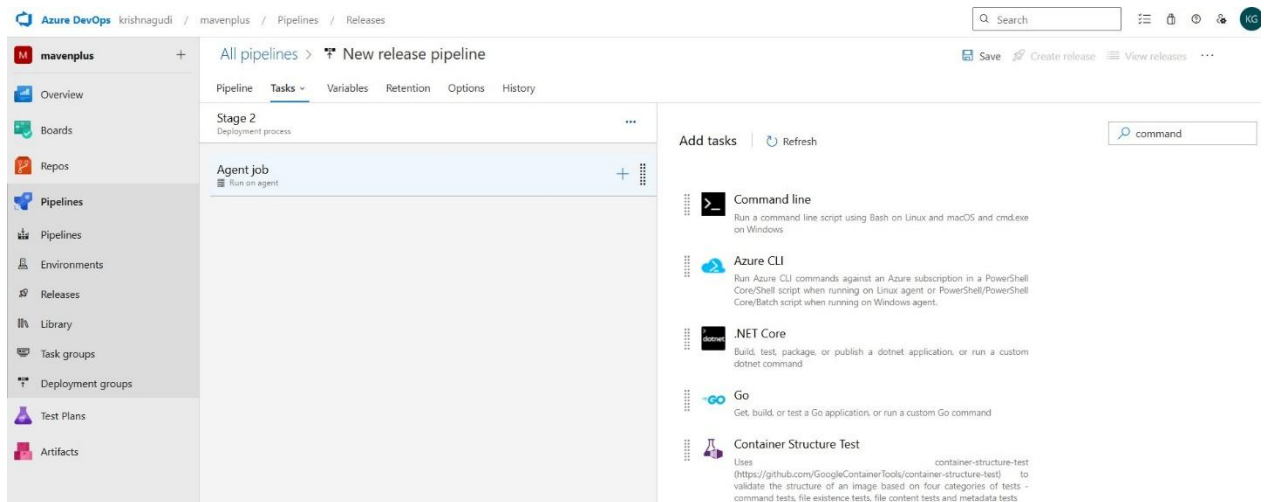


Fig 11.8: Adding deployment task in Tasks section

Step 9: In the command line task, enter deployment commands to simulate deployment or perform actual actions.

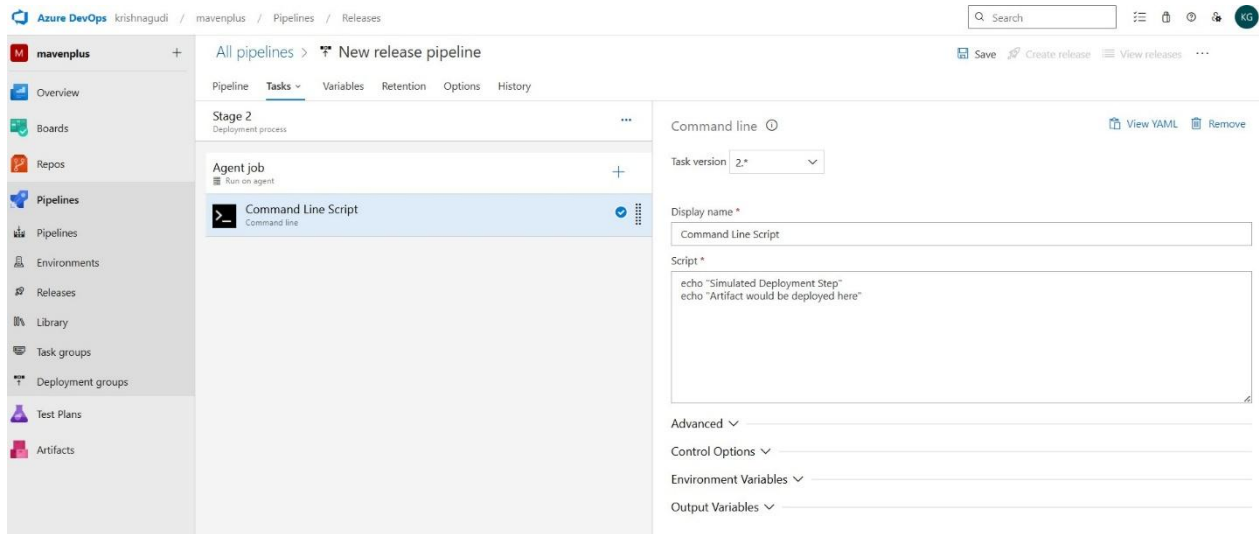


Fig 11.9: Entering deployment commands in command line task

Simulate Secrets (Key Vault):

Step 10: Click on '+ Variable group' to create a new group of shared pipeline variables.

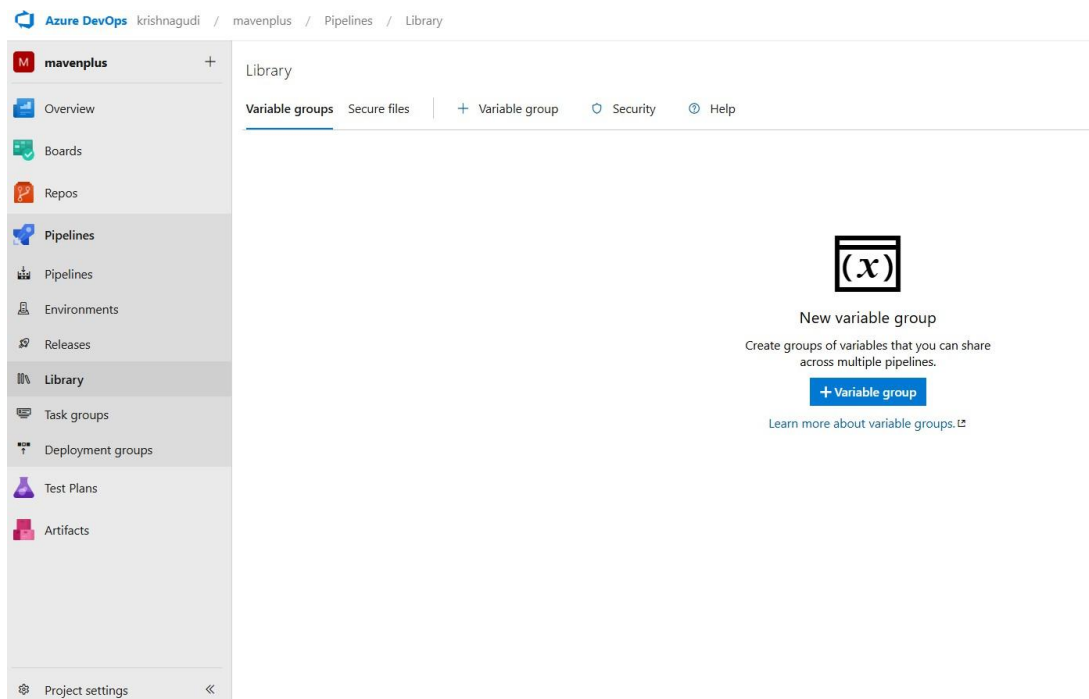


Fig 11.10: Creating a new Variable Group

Step 11: Enter a name and define secure variables (e.g., DB_PASSWORD) for use across multiple pipelines.

Azure DevOps krishnagudi / mavenplus / Pipelines / Library

mavenplus +

Overview
Boards
Repos
Pipelines
Pipelines
Environments
Releases
Library
Task groups
Deployment groups
Test Plans
Artifacts
Project settings <<

Library > New variable group 20-May*

Variable group | Save | Clone | Security | Pipeline permissions | Approvals and checks | Help

Properties

Variable group name
New variable group 20-May

Description

☒ Link secrets from an Azure key vault as variables ⓘ

Variables

Name ↑	Value	
name (required)		

+ Add

Fig 11.11: Defining secure variables in variable group

Step 12: Fill in variable name and value, and click Save to finalize the variable group setup.

Azure DevOps krishnagudi / mavenplus / Pipelines / Library

mavenplus +

Overview
Boards
Repos
Pipelines
Pipelines
Environments
Releases
Library
Task groups
Deployment groups
Test Plans
Artifacts
Project settings <<

Library > New variable group 20-May*

Variable group | Save | Clone | Security | Pipeline permissions | Approvals and checks | Help

Properties

Variable group name
New variable group 20-May

Description

☒ Link secrets from an Azure key vault as variables ⓘ

Variables

Name ↑	Value	
DB_PASSWORD	xyz123	

+ Add

Fig 11.12: Saving variable name and value in variable group

Link this variable group in your Release Pipeline:

Step 13: Select the created variable group and click 'Link' to use it in the release pipeline.

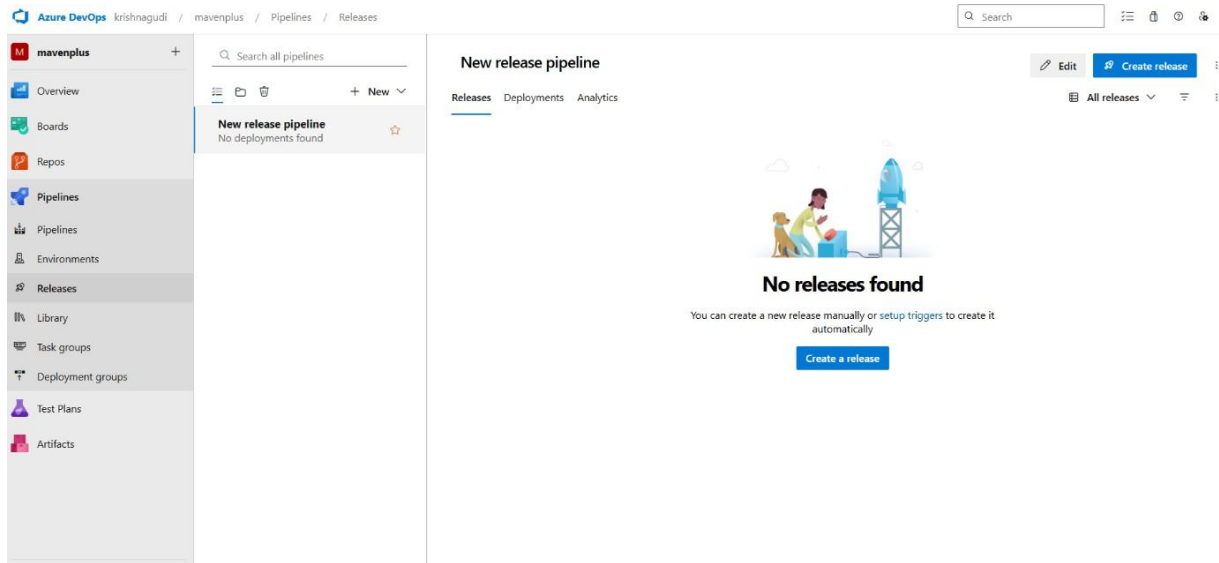


Fig 11.13: Linking variable group to release pipeline

Step 14: Click on 'Create a release' to trigger the release pipeline manually.

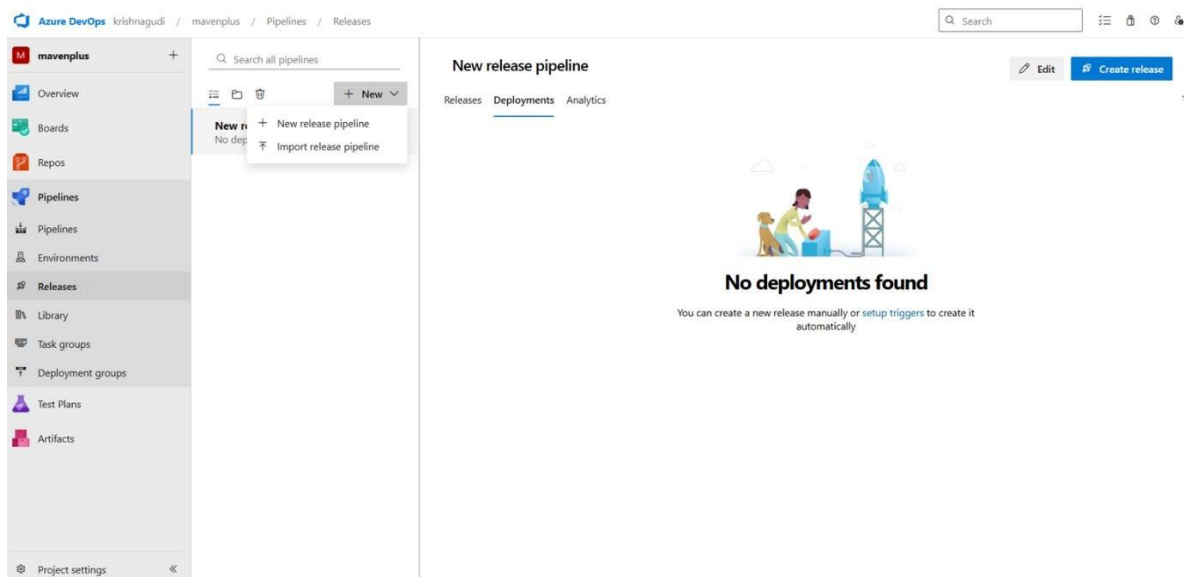


Fig 11.14: Creating a new release manually

Step 15: Click the '+ New' button to create or import a release pipeline when none exists.

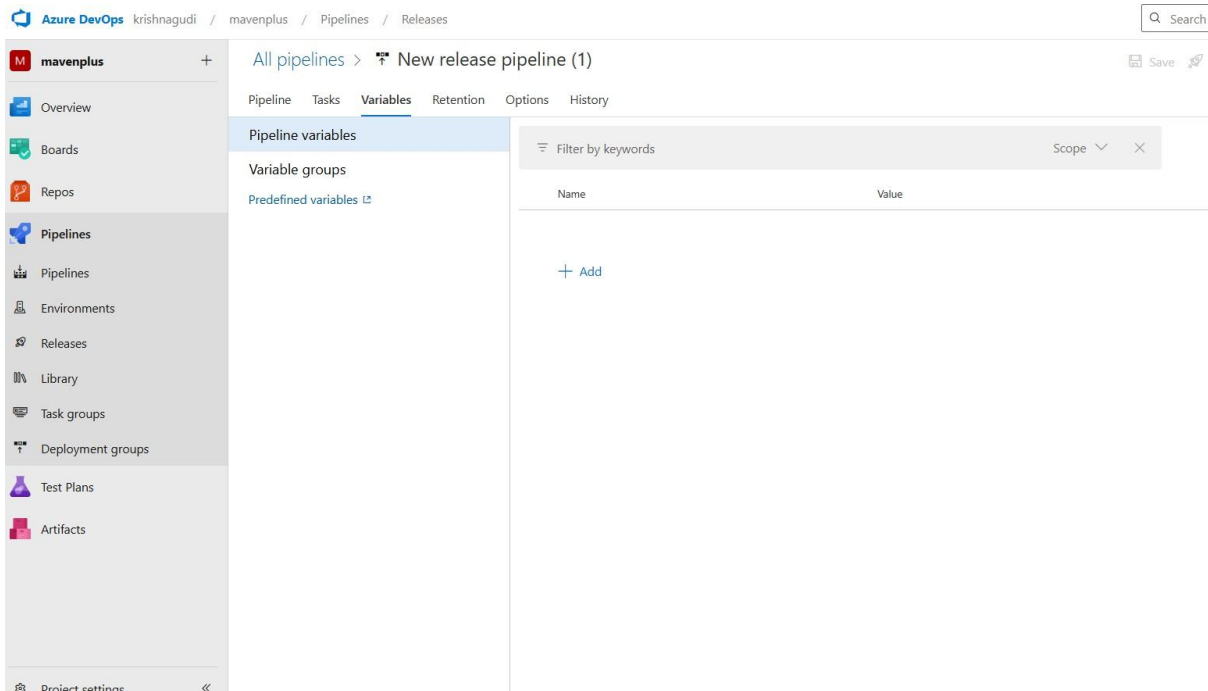


Fig 11.15: Creating or importing a release pipeline

Step 16: Click '+ Add' under Pipeline variables to define new custom key-value variables.

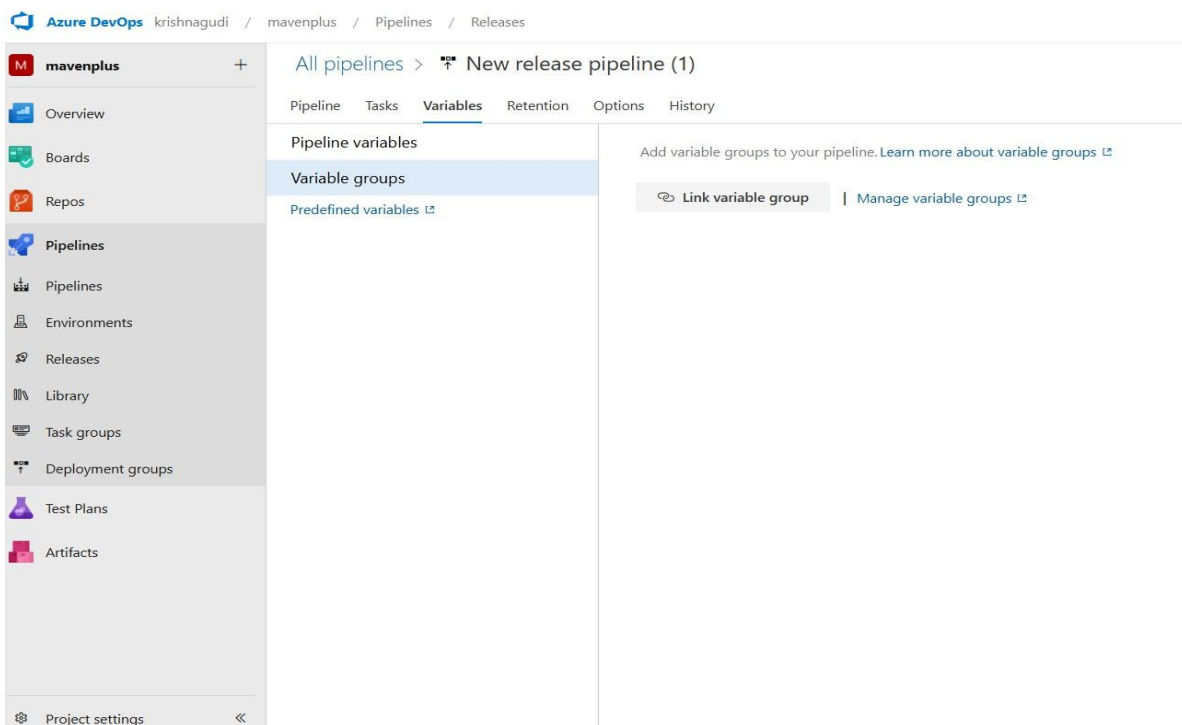


Fig 11.16: Adding custom key-value Pipeline variables

Step 17: Use 'Link variable group' to connect predefined variables to your release pipeline.

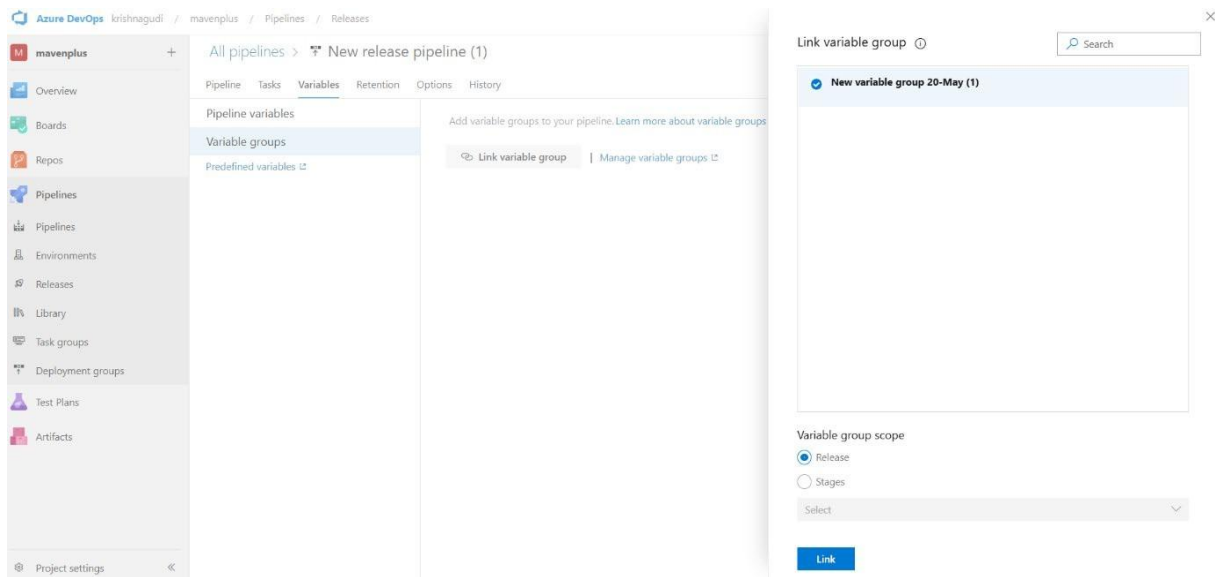


Fig 11.17: Linking variable group to release pipeline

“To overcome subscription-related limitations, a simulated deployment approach was adopted using command line scripts. This enabled demonstration of the complete release pipeline flow without requiring actual agent execution. It served as an effective workaround in a restricted environment.”

EXPERIMENT 12

Practical Exercise and Wake-Up: Build and Deploy a Complete DevOps Pipeline, Discussion on Best Practices and Q&A.

- Build Pipeline (CI) compiles the code, runs tests, and prepares an artifact (e.g., .jar, .zip) when code is pushed.
- Release Pipeline (CD) takes that artifact and deploys it to a target environment (e.g., Azure App Service).
- Together, it ensures smooth, continuous delivery from code commit to deployment with minimal manual steps.

Build Pipeline (CI) Setup:

Step 1: Go to Azure DevOps → Pipelines → Create Pipeline

Step 2: Select GitHub as the code repository and choose your sample Maven project.

Step 3: Choose **YAML** and create the following build steps:

- Checkout code
- Maven build task (mvn clean install)
- Publish artifact (e.g., JAR or ZIP)

Step 4: Commit and run the pipeline.

Step 5: If **Hosted Parallelism is not approved**, you may simulate steps using:

script: echo "Simulating Maven Build..."

script: echo "Simulating Artifact Publish..."

Release Pipeline (CD) Setup:

Step 6: Go to Pipelines → **Releases** → **New Pipeline**

Step 7: Add an Artifact → Select the build pipeline output.

Step 8: Add a Stage → Choose “**Empty Job**”

Step 9: In the Tasks tab, add a Command Line Script task:

echo "Simulating deployment to Azure App Service..."

echo "Deploying artifact: HelloWorld-1.0.jar"

Step 10: Save and create a release → Manually trigger it to run.

Step 11: Use screenshots to validate artifact linkage and simulation.

Simulating Secrets with Variable Groups:

Step 12: Go to Pipelines → Library → + Variable Group

Step 13: Name it (e.g., MySecretsGroup), add secure variables (e.g., DB_PASSWORD)

Step 14: Save the group and link it in your release pipeline (under Variables tab).

Discussion on Best Practices (Examples)

- Use version control for all YAML/pipeline files.
- Simulate builds when agents are unavailable.
- Store secrets securely in variable groups or Key Vault.
- Break pipelines into clear stages (build, test, deploy).
- Always review logs and test locally before release.

